Natural Language Processing

The Complete Guide — from raw text to transformers, taught end to end in sequence

Inside: 15 chapters • NLP pipeline • text cleaning & preprocessing (tokenisation, stop words, stemming, lemmatisation) • POS tagging (HMM & Viterbi), NER, chunking, parsing • one-hot, Bag of Words, unigrams/bigrams/trigrams, TF-IDF with fully worked numbers • n-gram language models, smoothing, perplexity • Word2Vec (CBOW, Skip-gram, negative sampling), Avg & TF-IDF-weighted Word2Vec, GloVe, FastText, Doc2Vec • text classification end-to-end (Naive Bayes maths, spam & sentiment projects) • similarity, BM25, topic models • RNN/LSTM/CNN for text • transformers, BERT fine-tuning, LLMs • metrics (BLEU, ROUGE…) • deployment • ~90 interview Q&A • cheat sheet

Prepared for Mansavi Aggarwal

Contents

1. Introduction to NLP **5**

1.1 What is Natural Language Processing? . . . 5

1.2 Levels of language analysis . . . 6

1.3 Why is NLP hard? . . . 6

1.4 A short history of NLP approaches . . . 6

1.5 Libraries & tools . . . 7

2. The End-to-End NLP Pipeline **8**

2.1 Step 1 — Data acquisition . . . 8

2.2 Step 2 — Text cleaning . . . 8

2.3 Step 3 — Pre-processing . . . 9

2.4 Step 4 — Feature engineering (text representation) . . . 9

2.5 Step 5 — Modelling: choose the right approach . . . 9

2.6 Step 6 — Evaluation . . . 9

2.7 Steps 7–8 — Deployment, monitoring, model update . . . 9

2.8 Worked example: what happens to one sentence . . . 10

3. Text Cleaning & Preprocessing **11**

3.1 Basic cleaning . . . 11

3.2 Tokenisation . . . 11

3.3 Stop-word removal . . . 12

3.4 Stemming . . . 13

3.5 Lemmatisation . . . 13

3.6 A reusable preprocessing function and the right order . . . 14

4. Linguistic Analysis: POS Tagging, NER, Chunking & Parsing **15**

4.1 Part-of-Speech (POS) tagging . . . 15

4.2 Named Entity Recognition (NER) . . . 16

4.3 Chunking (shallow parsing) . . . 17

4.4 Parsing . . . 17

4.5 Other linguistic tasks . . . 18

5. Text Representation I: One-Hot, Bag of Words, N-grams & TF-IDF **19**

5.1 Why do we need text representation? . . . 19

5.2 One-hot encoding . . . 19

5.3 Bag of Words (BoW) . . . 20

5.4 N-grams: unigrams, bigrams, trigrams . . . 20

5.5 TF-IDF (Term Frequency – Inverse Document Frequency) . . . 21

5.6 Other sparse techniques . . . 23
5.7 Comparison of classical representations . . . 23

6. Statistical Language Models 24

6.1 Language models . . . 24
6.2 Sparsity and smoothing . . . 24
6.3 Evaluating language models: perplexity . . . 25
6.4 Generating text with an n-gram model . . . 25

7. Word Embeddings: Word2Vec (CBOW & Skip-gram), Avg Word2Vec, GloVe, FastText, Doc2Vec 26

7.1 From sparse to dense: the idea of embeddings . . . 26
7.2 Word2Vec (Mikolov et al., Google, 2013) . . . 27
7.3 Average Word2Vec — from word vectors to document vectors . . . 29
7.4 GloVe (Global Vectors, Stanford 2014) . . . 30
7.5 FastText (Facebook AI, 2016) . . . 31
7.6 Doc2Vec (Paragraph Vectors) . . . 31
7.7 Evaluating and visualising embeddings . . . 31
7.8 Limitations of static embeddings . . . 31

8. Text Classification End-to-End 32

8.1 What is text classification? . . . 32
8.2 Algorithms . . . 32
8.3 Project 1 — SMS spam detection (end-to-end) . . . 34
8.4 Project 2 — Sentiment analysis . . . 34
8.5 Handling real-world issues . . . 35
8.6 Evaluation metrics for classification . . . 35
8.7 Error analysis checklist . . . 36

9. Text Similarity, Search, Topic Modelling & Clustering 37

9.1 Similarity measures . . . 37
9.2 Information retrieval & BM25 . . . 38
9.3 Topic modelling . . . 38
9.4 Document clustering and keyword extraction . . . 39

10. Deep Learning for NLP 40

10.1 Preparing text for neural networks . . . 40
10.2 The Embedding layer . . . 40
10.3 RNN, LSTM, GRU and Bi-LSTM for text . . . 41
10.4 1-D CNN for text . . . 41
10.5 Sequence-to-sequence, attention and sequence labelling . . . 41

11. Transformers & Modern NLP **42**

11.1 Self-attention in one page . . . 42
11.2 Subword tokenisation . . . 43
11.3 The BERT, GPT and T5 families . . . 44
11.4 Hugging Face pipelines (zero code training) . . . 44
11.5 Fine-tuning BERT for text classification . . . 44
11.6 Sentence embeddings and LLMs for NLP tasks . . . 45

12. NLP Tasks & Evaluation Metrics **46**

12.1 Task catalogue . . . 46
12.2 Generation metrics . . . 47

13. Deploying NLP Models **48**

13.1 Save everything that transforms text . . . 48
13.2 Serve with FastAPI . . . 48
13.3 Production concerns . . . 48

14. Interview Questions & Answers **49**

15. Cheat Sheet & Learning Sequence **55**

15.1 Formulas . . . 55
15.2 Which technique when? . . . 56
15.3 Recommended learning sequence . . . 56

CHAPTER 1

Introduction to NLP

What NLP is • applications • levels of language • why NLP is hard • history: rules → statistics → neural → transformers → LLMs • NLP vs NLU vs NLG • tools & libraries

1.1 What is Natural Language Processing?

Natural Language Processing (NLP) is the field of AI that enables computers to **understand, interpret, and generate human language** — text and speech. Computers work with numbers, so the core challenge of NLP is turning messy, ambiguous, context-dependent language into numerical representations that models can learn from, and turning model outputs back into language.

Term	Meaning	Example
NLP	the whole field	everything below
NLU (understanding)	extract meaning, intent, entities	'Book a cab to the airport' → intent=book_cab, dest=airport
NLG (generation)	produce fluent text	summaries, chatbot replies, reports
Computational linguistics	linguistics-driven study of language with computers	parsing theory, morphology

Applications you use every day

Application	NLP task(s)
Gmail spam filter, ticket routing	text classification
Product review analysis	sentiment analysis, aspect extraction
Google Search	information retrieval, query understanding, ranking
Autocomplete, keyboard suggestions	language modelling
Google Translate	machine translation (seq2seq / transformers)
Alexa / Siri	speech recognition + intent detection + NLG + TTS
Resume parsing, KYC extraction	named entity recognition, information extraction
ChatGPT, Gemini, Copilot	large language models (generation, QA, reasoning)
News digests	summarisation
Grammarly	spell/grammar correction

1.2 Levels of language analysis

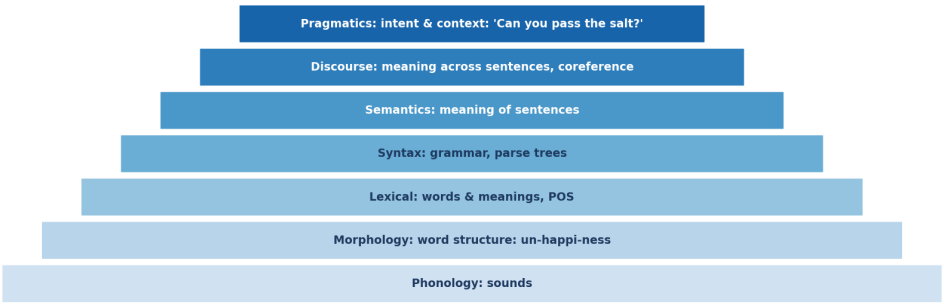


Figure: From sounds up to intent — each level needs different techniques.

1.3 Why is NLP hard?

Challenge	Example
Lexical ambiguity	'bank' = river bank or financial bank
Syntactic ambiguity	'I saw the man with the telescope' — who has the telescope?
Semantic / pragmatic ambiguity	'Can you open the window?' is a request, not a question about ability
Coreference	'Riya told Meera she won' — who won?
Sarcasm / irony	'Great, my flight is delayed again!' (negative)
Variation	slang, spelling mistakes, code-mixing ('kal meeting hai na?'), emojis, domain jargon
Context & world knowledge	'The trophy didn't fit in the suitcase because it was too big' — 'it' = trophy
Sparsity & long tail	most words are rare (Zipf's law); new words keep appearing (OOV)
Low-resource languages	little labelled data for many Indian languages

1.4 A short history of NLP approaches

Era	Approach	Examples	Limitation
1950s–1980s	Rule-based / symbolic: hand-written grammars, regex, dictionaries	ELIZA chatbot, expert systems	brittle, doesn't scale
1990s–2000s	Statistical ML on hand-crafted features	n-gram LMs, HMM POS tagging, Naive Bayes, CRF NER, phrase-based MT	feature engineering heavy, sparse features
2013–2017	Neural + word embeddings	Word2Vec, GloVe, RNN/LSTM, seq2seq + attention	static embeddings, sequential training
2018–2020	Pre-trained transformers + fine-tuning	BERT, GPT-2, RoBERTa, T5	need task-specific fine-tuning
2020–now	Large language models, prompting, RAG, agents	GPT-4-class, Gemini, Claude, LLaMA	cost, hallucination, evaluation

Interview tip: This book follows that same path: you will first learn classical techniques (preprocessing, BoW, TF-IDF, n-grams, Naive Bayes) because they are fast, interpretable baselines still used in production and asked in every interview — then embeddings, deep learning and transformers.

1.5 Libraries & tools

Library	Strengths	Typical use
NLTK	teaching toolkit; tokenisers, stemmers, WordNet, corpora, POS/NER (classic)	learning, preprocessing
spaCy	fast industrial pipelines: tokenisation, lemmas, POS, dependency parse, NER, custom pipelines	production preprocessing & extraction
scikit-learn	CountVectorizer, TfidfVectorizer, classifiers, pipelines, metrics	classical text classification
Gensim	Word2Vec, FastText, Doc2Vec, LDA, similarity	embeddings & topic models
Hugging Face transformers / datasets / tokenizers	pre-trained models, fine-tuning, fast tokenisers	modern NLP
sentence-transformers	sentence embeddings, cross-encoders	semantic search, RAG
TextBlob / VADER	simple sentiment & polarity	quick baselines
regex (re), BeautifulSoup, emoji, contractions, langdetect	cleaning	preprocessing
Indic NLP: indic-nlp-library, iNLTK, IndicBERT, MuRIL	Indian languages	regional NLP

setup

```
pip install nltk spacy scikit-learn gensim transformers datasets sentence-transformers
python -m spacy download en_core_web_sm
python -c "import nltk; [nltk.download(p) for p in ['punkt','punkt_tab','stopwords','wordnet','omw-1.4','averaged_perceptron
        _tagger_eng','maxent_ne_chunker_tab','words']]"
```

CHAPTER 2

The End-to-End NLP Pipeline

Data acquisition • text cleaning • preprocessing • feature engineering • modelling • evaluation • deployment • monitoring • heuristics vs ML vs DL • a worked example

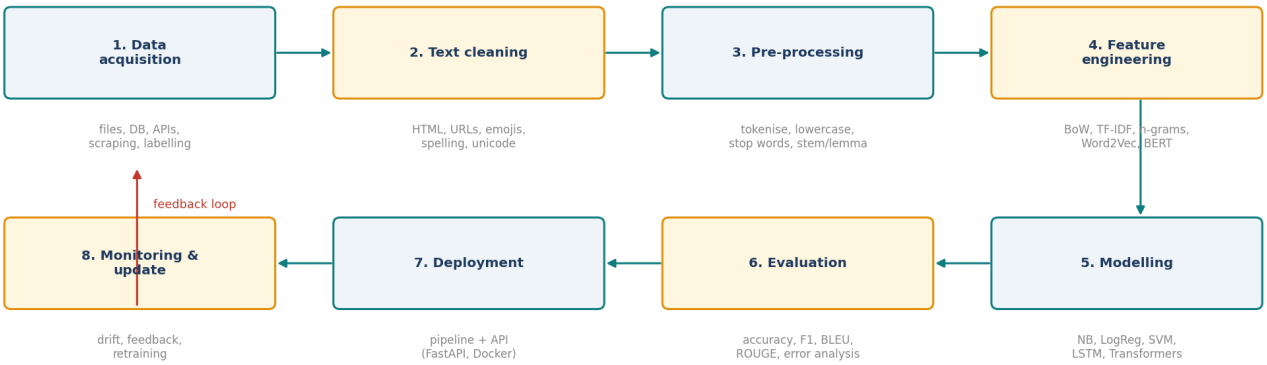


Figure: The NLP project life cycle. It is iterative: error analysis sends you back to earlier steps.

2.1 Step 1 — Data acquisition

Situation	Strategy
Data available in-house	databases, CSV, logs, tickets, emails, chat transcripts (check privacy/consent)
Public data	Kaggle, Hugging Face datasets, UCI, government portals
Web data	scraping (BeautifulSoup, Scrapy) or APIs — respect robots.txt and terms
No labels	manual annotation (Label Studio, Prodigy), weak supervision with heuristics, LLM-assisted labelling + human review, active learning
Too little data	data augmentation: synonym replacement (WordNet), back-translation, random insertion/deletion/swap (EDA), paraphrasing with LLMs
Imbalanced classes	collect more minority examples, resample, class weights

2.2 Step 2 — Text cleaning

Remove noise that carries no meaning for the task: HTML tags, URLs, email addresses, special characters, extra whitespace, encoding issues, boilerplate (signatures, disclaimers), duplicated text; handle emojis, spelling mistakes and unicode normalisation. Details in Chapter 3.

2.3 Step 3 — Pre-processing

Basic	Advanced (task dependent)
tokenisation, lowercasing, stop-word removal, punctuation/digit removal, stemming or lemmatisation	POS tagging, NER, parsing, coreference, language detection, chunking

2.4 Step 4 — Feature engineering (text representation)

Family	Techniques	Chapter
Frequency-based (sparse)	one-hot, Bag of Words, n-grams, TF-IDF	5
Prediction-based (dense)	Word2Vec (CBOW, Skip-gram), GloVe, FastText, Avg Word2Vec, Doc2Vec	7
Contextual	ELMo, BERT, sentence-transformers, LLM embeddings	11
Hand-crafted	length, # of exclamation marks, capital ratio, sentiment lexicon scores, presence of URLs	8

2.5 Step 5 — Modelling: choose the right approach

Approach	When	Examples
Heuristics / rules	no data, very clear patterns, need 100% control	regex for ticket IDs, keyword lists, VADER lexicon
Classical ML	small–medium labelled data, need speed & interpretability	Naive Bayes, Logistic Regression, SVM on TF-IDF
Deep learning	more data, complex patterns, sequence labelling	LSTM, CNN, fine-tuned BERT
LLMs / prompting	little labelled data, many tasks, generation	zero/few-shot classification, extraction, summarisation

2.6 Step 6 — Evaluation

Intrinsic evaluation measures the model on the NLP task (accuracy, F1, perplexity, BLEU, ROUGE). **Extrinsic** evaluation measures business impact (tickets auto-resolved, CSAT, time saved). Always do **error analysis**: read misclassified examples and group them (negation, sarcasm, rare words, label noise).

2.7 Steps 7–8 — Deployment, monitoring, model update

- Save **the whole pipeline** (cleaning + vectoriser + model) so production uses identical preprocessing.
- Serve via an API (FastAPI) in a Docker container; batch scoring for offline jobs.
- Monitor input drift (new vocabulary, new topics, OOV rate), prediction distribution and accuracy on fresh labels; retrain periodically.

2.8 Worked example: what happens to one sentence

Stage	Output
Raw	The FOOD wasn't good!!! ☹ Visit http://x.co
Cleaned	The FOOD wasn't good!!! angry
Lower + contractions	the food was not good!!! angry
Tokenised	['the', 'food', 'was', 'not', 'good', '!', '!', '!', 'angry']
Stop words removed (keep 'not'!)	['food', 'not', 'good', 'angry']
Lemmatised	['food', 'not', 'good', 'angry']
Uni+bi-gram TF-IDF	sparse vector over features like 'food', 'not good', 'good angry' ...
Model output	label = negative, p = 0.93

CHAPTER 3

Text Cleaning & Preprocessing

Lowercasing • HTML/URL/punctuation removal • contractions • spelling • emojis • unicode • tokenisation (sentence, word, subword, character) • stop words • stemming (Porter, Snowball, Lancaster) • lemmatisation • normalisation • recommended order • code

3.1 Basic cleaning

Operation	Why	Code / caution
Lowercasing	'Good' and 'good' become one feature → smaller vocab	text.lower(); keep case for NER ('Apple' vs 'apple')
Remove HTML tags	markup is noise	BeautifulSoup(text, 'html.parser').get_text() or re.sub(r'<.*?>', '', t)
Remove URLs / emails	rarely useful	re.sub(r'https?://\S+|www\.\S+', '', t)
Remove punctuation	reduces noise for BoW	t.translate(str.maketrans('', '', string.punctuation)); keep '!' or '?' if useful for sentiment
Remove / replace numbers	often noise; sometimes important (prices)	re.sub(r'\d+', '<NUM>', t)
Expand contractions	"can't" → 'can not' (keeps negation)	contractions.fix(t)
Chat words / slang	'u' → 'you', 'gr8' → 'great', 'idk' → 'I do not know'	custom dictionary
Spelling correction	'recieve' → 'receive'	TextBlob(t).correct(), SymSpell — slow; may 'fix' names
Emojis	carry sentiment	emoji.demojize('😡') → ':angry_face:' or remove
Unicode normalisation	'café' vs 'café' (composed vs decomposed); fancy quotes	unicodedata.normalize('NFKC', t)
Whitespace	collapse multiple spaces/newlines	re.sub(r'\s+', ' ', t).strip()

3.2 Tokenisation

Tokenisation splits text into units called **tokens**. It is the first and most important preprocessing step because every later representation is built on tokens. The set of unique tokens is the **vocabulary** (V); the full text collection is the **corpus**; one item (review, email) is a **document**.

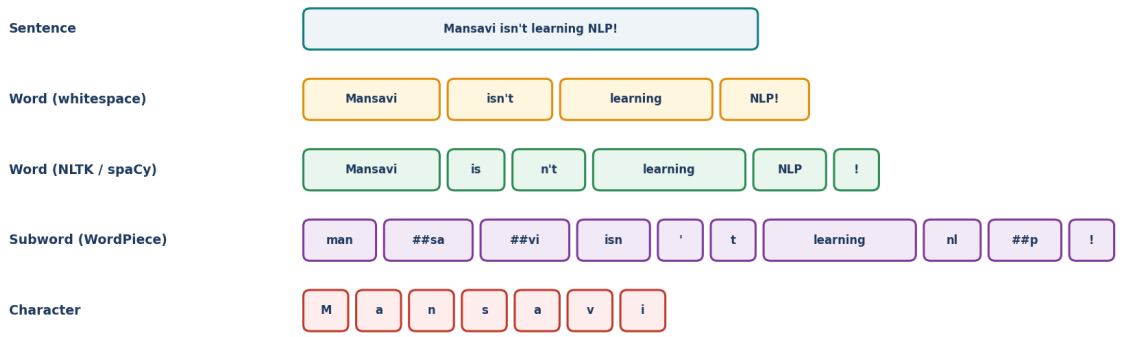


Figure: The same sentence tokenised at different granularities.

Type	Pros	Cons	Used by
Sentence	needed for summarisation, per-sentence analysis	abbreviations ('Dr.') break naive splitting	nltk.sent_tokenize, spaCy doc.sents
Word	intuitive, interpretable features	huge vocab, OOV words, punctuation and contractions need rules	split(), nltk.word_tokenize, spaCy
Subword (BPE, WordPiece, Unigram)	small vocab, no OOV, handles morphology	tokens less interpretable	BERT, GPT, LLaMA
Character	tiny vocab, no OOV, robust to typos	very long sequences, less meaning per token	char-CNNs, some OCR/spelling models

tokenisation.py

```
import re, nltk, spacy
text = "Dr. Rao isn't in Bengaluru today. He'll join at 3 p.m.!"
text.split()                                  # ['Dr.', 'Rao', "isn't", ...]  naive
re.findall(r"\w+", text)                      # ['Dr', 'Rao', 'isn', 't', ...]  regex
nltk.sent_tokenize(text)                      # ['Dr. Rao isn't in Bengaluru today.', "He'll join at 3 p.m.!"]
nltk.word_tokenize(text)                      # ['Dr.', 'Rao', 'is', "n't", 'in', 'Bengaluru', ...]
from nltk.tokenize import TweetTokenizer, RegexpTokenizer
TweetTokenizer().tokenize("@raj loved it!!! #NLP :)")       # keeps @mentions, #hashtags, emoticons
nlp = spacy.load("en_core_web_sm"); [t.text for t in nlp(text)]
```

3.3 Stop-word removal

Stop words are very frequent function words (the, is, at, of, and) carrying little topical information. Removing them shrinks the vocabulary and focuses BoW/TF-IDF on content words. By **Zipf's law**, a handful of words make up a large share of all tokens:

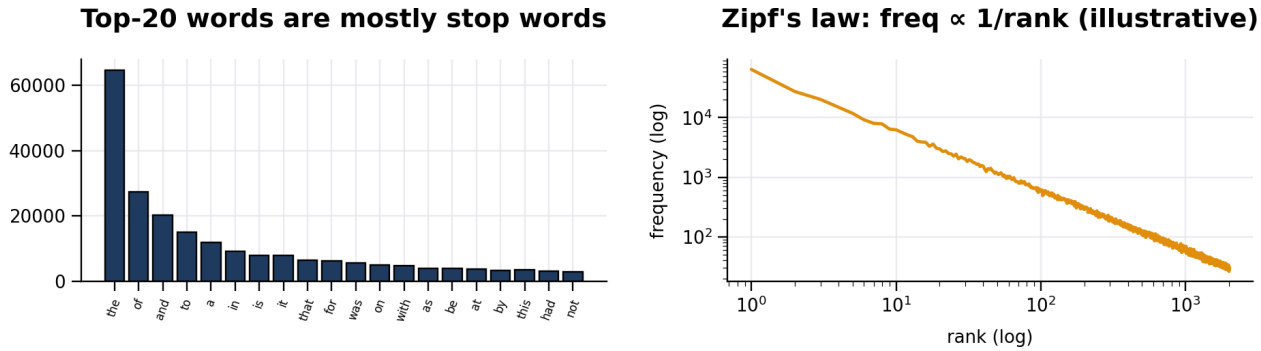


Figure: Word frequency is extremely skewed: frequency ≈ C / rank.

Watch out: Do NOT blindly remove stop words for sentiment ('not', 'no', 'never' flip meaning), question answering ('who', 'where'), machine translation, or any transformer model. Customise the list: remove negations from it, add domain stop words ('please', 'regards', 'ticket').

stopwords.py

```
from nltk.corpus import stopwords
stops = set(stopwords.words("english")) - {"not", "no", "nor", "never"}    # keep negations
tokens = [w for w in tokens if w not in stops]
# sklearn: TfidfVectorizer(stop_words="english")  |  spaCy: token.is_stop
```

3.4 Stemming

Stemming chops affixes using heuristic rules to reduce words to a **stem**, which may not be a real word. It is fast and language-rule based, reducing vocabulary size so 'connect', 'connected', 'connection' become one feature.

Stemmer	Behaviour	Example: studies / running / generously / fairly
Porter (1980)	5 phases of suffix-rewrite rules; moderate	studi / run / gener / fairli
Snowball (Porter2)	improved Porter; multiple languages	studi / run / generous / fair
Lancaster	very aggressive (over-stemming)	study / run / gen / fair
RegexpStemmer	your own suffix regex	custom

stemming.py

```
from nltk.stem import PorterStemmer, SnowballStemmer, LancasterStemmer
ps, sb = PorterStemmer(), SnowballStemmer("english")
[ps.stem(w) for w in ["connect", "connected", "connection", "connecting"]]   # ['connect']*4
ps.stem("university"), ps.stem("universe")     # both 'univers' → OVER-stemming (different meanings merged)
ps.stem("data"), ps.stem("datum")              # 'data', 'datum' → UNDER-stemming (same meaning kept apart)
```

3.5 Lemmatisation

Lemmatisation maps a word to its dictionary form (**lemma**) using vocabulary and morphological analysis, usually needing the **part of speech**: 'better' (adj) → 'good', 'was' → 'be', 'mice' → 'mouse'. Output is always a real word; it is slower than stemming.

lemmatisation.py

```
from nltk.stem import WordNetLemmatizer
lem = WordNetLemmatizer()
lem.lemmatize("running")            # 'running'  (default POS = noun!)
lem.lemmatize("running", pos="v")   # 'run'
lem.lemmatize("better", pos="a")    # 'good'
# spaCy picks POS automatically:
[(t.text, t.lemma_) for t in nlp("The mice were running better")]   # mouse, be, run, well
```

	Stemming	Lemmatisation
Method	rule-based suffix stripping	dictionary + POS + morphology
Output	may be non-word ('studi')	valid word ('study')
Speed	fast	slower
Accuracy	lower (over/under-stemming)	higher

	Stemming	Lemmatisation
Use	search engines, large-scale BoW where speed matters	chatbots, QA, when meaning/readability matters

3.6 A reusable preprocessing function and the right order

preprocess.py

```
import re, string, unicodedata, contractions, emoji
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
STOPS = set(stopwords.words("english")) - {"not", "no", "never", "nor"}
LEM = WordNetLemmatizer()

def preprocess(text: str, remove_stop=True, lemmatize=True) -> str:
    text = unicodedata.normalize("NFKC", text)
    text = re.sub(r"<.*?>", " ", text)                          # HTML
    text = re.sub(r"https?://\S+|www\.\S+|\S+@\S+", " ", text)  # URLs, emails
    text = emoji.demojize(text, delimiters=(" ", " "))          # 😠 → angry_face
    text = contractions.fix(text).lower()                       # wasn't → was not
    text = re.sub(r"[^a-z\s_]", " ", text)                       # punctuation & digits
    tokens = text.split()
    if remove_stop: tokens = [t for t in tokens if t not in STOPS]
    if lemmatize: tokens = [LEM.lemmatize(t) for t in tokens]
    return " ".join(tokens)

preprocess("<p>The FOOD wasn't good!!! 😠 http://x.co</p>")        # 'food not good angry_face'
df["clean"] = df["text"].apply(preprocess)
```

1. Normalise encoding → remove HTML/URLs → handle emojis → expand contractions → lowercase.
2. Remove punctuation/digits (task-dependent).
3. Tokenise.
4. Remove stop words (keeping negations).
5. Stem or lemmatise (not both).
6. Re-join or keep tokens depending on the vectoriser.

> **Interview tip:** Golden rule: preprocessing must match the model. Classical models (BoW/TF-IDF/NB) benefit from heavy cleaning; transformer models need the raw text passed through their own tokenizer — at most light cleaning (HTML/URLs).

CHAPTER 4

Linguistic Analysis: POS Tagging, NER, Chunking & Parsing

POS tag sets • rule-based vs statistical tagging • HMM & Viterbi math • NLTK & spaCy • NER & BIO scheme • CRF • chunking • dependency & constituency parsing • coreference

4.1 Part-of-Speech (POS) tagging

POS tagging assigns each token its grammatical category (noun, verb, adjective…). It is used for lemmatisation (needs POS), information extraction (noun phrases), word-sense disambiguation ('book' noun vs verb), parsing, and as features for NER/classification.

Penn Treebank tag	Meaning	Universal tag
NN / NNS	noun singular / plural	NOUN
NNP / NNPS	proper noun	PROPN
VB / VBD / VBG / VBN / VBP / VBZ	verb base / past / gerund / past participle / non-3rd present / 3rd person present	VERB
JJ / JJR / JJS	adjective / comparative / superlative	ADJ
RB	adverb	ADV
PRP / PRP$	personal / possessive pronoun	PRON
DT	determiner	DET
IN	preposition / subordinating conjunction	ADP
CC	coordinating conjunction	CCONJ
MD	modal (can, will)	AUX
CD	cardinal number	NUM

pos.py

```
import nltk
nltk.pos_tag(nltk.word_tokenize("They can fish in the river"))
# [('They','PRP'), ('can','MD'), ('fish','VB'), ('in','IN'), ('the','DT'), ('river','NN')]
nltk.pos_tag(["book"]), nltk.pos_tag(nltk.word_tokenize("Please book a flight"))    # ambiguity resolved by context
doc = nlp("Apple is looking at buying a U.K. startup")
[(t.text, t.pos_, t.tag_, t.lemma_) for t in doc]          # universal POS, fine tag, lemma
```

How taggers work

Approach	Idea
Rule-based	lexicon + hand-written rules ('word after DT is likely NN')
Most-frequent-tag baseline	assign each word its most common tag (~90% accuracy!)
Hidden Markov Model	generative sequence model with transition and emission probabilities, decoded by Viterbi

Approach	Idea
MaxEnt / Perceptron (NLTK default), CRF	discriminative models using rich features (suffixes, capitalisation, neighbours)
Neural (BiLSTM-CRF, transformers)	contextual embeddings → ~97–98% accuracy (spaCy, Stanza)

Hidden Markov Model (HMM) for POS tagging

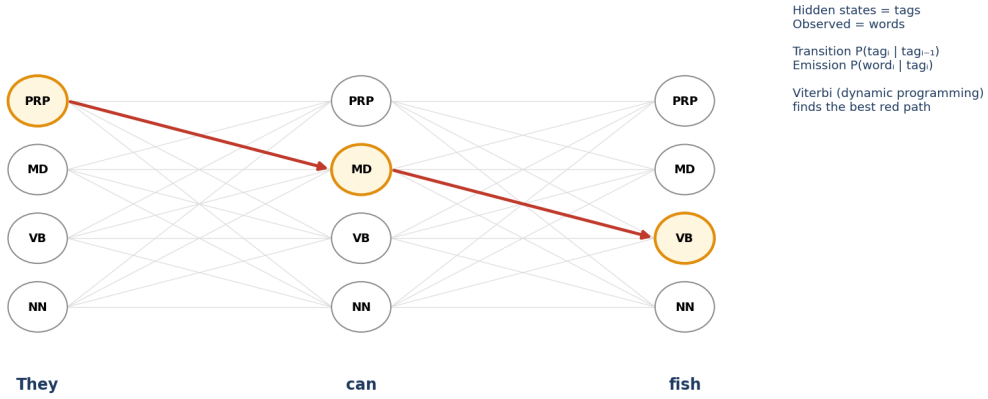


Figure: HMM trellis for 'They can fish': Viterbi picks the most probable tag path.

$$\hat{t}_{1:n} = \arg\max_{t_{1:n}} P(t_{1:n} \mid w_{1:n}) = \arg\max_{t_{1:n}} \prod_{i=1}^{n} P(w_i \mid t_i)\, P(t_i \mid t_{i-1})$$

emission probability × transition probability

$$P(t_i \mid t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})} \qquad P(w_i \mid t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

both estimated by counting in a tagged corpus (MLE)

$$v_i(t) = \max_{t'} v_{i-1}(t')\, P(t \mid t')\, P(w_i \mid t)$$

Viterbi recursion: best score of any path ending in tag t at position i; keep back-pointers; complexity $O(n \cdot T^2)$

Assumptions: (1) **Markov** — a tag depends only on the previous tag (bigram HMM); (2) **output independence** — a word depends only on its own tag. Brute force over all T^n tag sequences is impossible; Viterbi dynamic programming makes it linear in n.

4.2 Named Entity Recognition (NER)

NER finds spans of text that refer to entities and classifies them: PERSON, ORG, LOC/GPE, DATE, MONEY, PRODUCT, plus custom domain entities (ACCOUNT_NO, DRUG, SKILL). It is a **sequence labelling** task, usually encoded with the **BIO** scheme (Begin, Inside, Outside).



Figure: BIO tags: B- starts an entity, I- continues it, O is outside any entity.

ner.py

```
doc = nlp("Sundar Pichai visited Infosys in New Delhi on Monday for $5 million deal")
[(e.text, e.label_) for e in doc.ents]
# [('Sundar Pichai','PERSON'), ('Infosys','ORG'), ('New Delhi','GPE'), ('Monday','DATE'), ('$5 million','MONEY')]
from spacy import displacy; displacy.render(doc, style="ent")      # highlighted view in Jupyter
nltk.ne_chunk(nltk.pos_tag(nltk.word_tokenize("Sundar Pichai works at Google")))   # NLTK classic chunker

from transformers import pipeline
ner = pipeline("ner", model="dslim/bert-base-NER", aggregation_strategy="simple"); ner("Mansavi joined LTIMindtree in
        Bengaluru")
```

Approach	Notes
Rule / gazetteer	regex for IDs, dictionaries of names; spaCy EntityRuler
CRF (Conditional Random Field)	discriminative sequence model with features (word shape, prefix/suffix, POS, neighbours); learns valid tag transitions (I-PER cannot follow B-ORG)
BiLSTM-CRF	neural features + CRF output layer
Fine-tuned BERT (token classification)	state of the art; labels aligned to sub-word tokens
LLM extraction	prompt + JSON schema; good for few-shot custom entities

Evaluation is **entity-level** (seqeval): an entity counts as correct only if both span boundaries and type match exactly → precision, recall, F1 per entity type.

4.3 Chunking (shallow parsing)

Groups tokens into non-overlapping phrases — mostly noun phrases (NP) — without building a full tree. Example rule with NLTK RegexpParser: *NP: {<DT>?<JJ>*<NN.*>+}* captures 'the quick brown fox'. spaCy offers doc.noun_chunks. Useful for keyphrase extraction.

4.4 Parsing

Dependency parsing

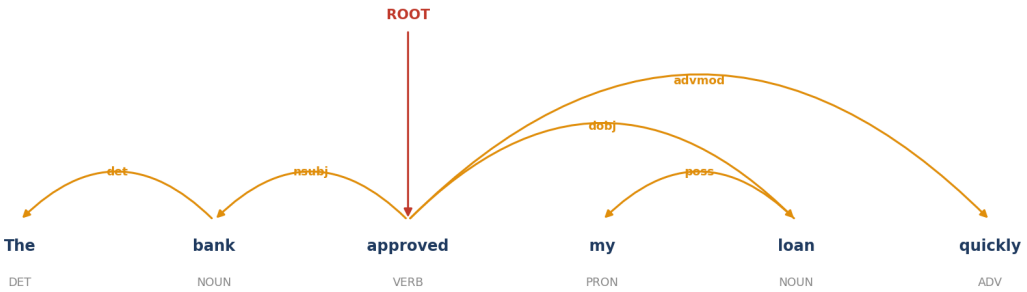


Figure: Dependency parse: arrows go from head to dependent with a grammatical relation label.

Each word is linked to its **head** with a labelled relation (nsubj = subject, dobj = object, amod = adjective modifier, det, prep, advmod…); the main verb is the ROOT. Used for relation extraction ('who did what to whom'), aspect-based sentiment ('battery' ← amod ← 'poor'), and question answering.

dependency.py

```
for t in nlp("The bank approved my loan quickly"):
    print(t.text, t.dep_, "→", t.head.text)
# The det → bank | bank nsubj → approved | approved ROOT → approved | my poss → loan | loan dobj → approved | quickly
        advmod → approved
```

Constituency parsing

Builds a phrase-structure tree using a grammar (S → NP VP, NP → DT NN …), e.g. (S (NP The bank) (VP approved (NP my loan) (ADVP quickly))). Algorithms: CKY with probabilistic context-free grammars (PCFG), and neural parsers (Berkeley Neural Parser).

4.5 Other linguistic tasks

Task	Description
Coreference resolution	link mentions of the same entity: 'Riya … she … her'
Word sense disambiguation	pick the correct meaning of 'bank' (Lesk algorithm with WordNet, or contextual embeddings)
Morphological analysis	split into morphemes; crucial for Indian languages
Language detection	langdetect / fastText lid model
Sentence boundary detection	split sentences robustly

CHAPTER 5

Text Representation I: One-Hot, Bag of Words, N-grams & TF-IDF

Why vectorise • one-hot encoding • Bag of Words (count & binary) • unigrams, bigrams, trigrams • TF-IDF maths with a full worked example • sklearn parameters • hashing • comparison & limitations

5.1 Why do we need text representation?

Machine-learning models take **fixed-length numeric vectors**, but text is variable-length symbols. **Vectorisation** (feature extraction) maps text to numbers while keeping as much meaning as possible. A good representation should place similar texts close together and keep task-relevant signal (e.g. negation for sentiment).

Running corpus used throughout this chapter (after lowercasing):

Doc	Text
D1	the food was good
D2	the food was bad
D3	good food good service
D4	bad service

Vocabulary (sorted alphabetically, as sklearn does): **bad, food, good, service, the, was** → V = 6.

5.2 One-hot encoding

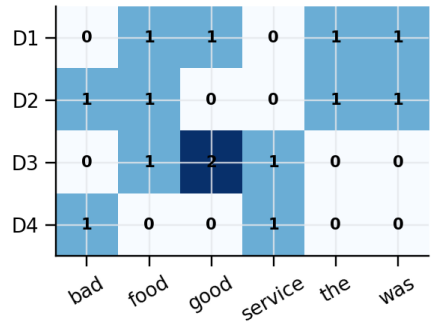
Each word is a V-dimensional vector with a single 1 at its vocabulary index: food = [0,1,0,0,0,0], good = [0,0,1,0,0,0]. A document becomes a sequence of such vectors (a matrix of shape length × V).

Pros	Cons
simple, intuitive	sparse and huge (V can be 100k+); no notion of similarity — every pair of words is orthogonal (cos = 0, 'good' is as far from 'great' as from 'banana'); variable size per document; out-of-vocabulary (OOV) words can't be represented

5.3 Bag of Words (BoW)

Bag of Words represents a document by the **count** of each vocabulary word, ignoring grammar and word order — like emptying the words into a bag and counting them.

Bag of Words (CountVectorizer)



TF-IDF (sklearn: smooth idf + L2 norm)

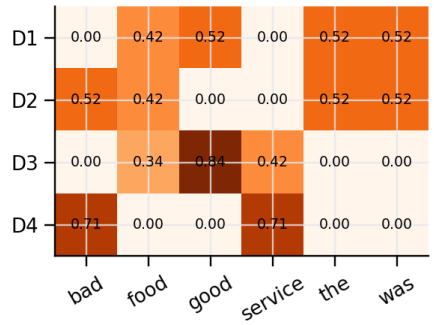


Figure: Left: BoW count matrix of the corpus (computed with CountVectorizer). Right: TF-IDF matrix of the same corpus.

Doc	bad	food	good	service	the	was
D1	0	1	1	0	1	1
D2	1	1	0	0	1	1
D3	0	1	2	1	0	0
D4	1	0	0	1	0	0

Note D3 has good = 2. **Binary BoW** (CountVectorizer(binary=True)) records only presence (1/0) — often better for short texts and Bernoulli Naive Bayes.

bow.py

```
from sklearn.feature_extraction.text import CountVectorizer
docs = ["the food was good", "the food was bad", "good food good service", "bad service"]
cv = CountVectorizer()                      # lowercase=True, token_pattern=r"(?u)\b\w\w+\b" (drops 1-char tokens!)
X = cv.fit_transform(docs)                  # scipy sparse matrix (4 × 6)
cv.get_feature_names_out()                  # ['bad' 'food' 'good' 'service' 'the' 'was']
X.toarray(); cv.vocabulary_                 # {'the': 4, 'food': 1, ...}
cv.transform(["good bad pizza"]).toarray()  # [[1 0 1 0 0 0]]  ← 'pizza' is OOV → ignored
```

Advantages	Disadvantages
simple, fast, interpretable; strong baseline with Naive Bayes / Logistic Regression; fixed-size vectors	ignores word order ('dog bites man' = 'man bites dog'); sparse high-dimensional; no semantics (synonyms are unrelated); frequent words dominate; OOV words dropped

5.4 N-grams: unigrams, bigrams, trigrams

An **n-gram** is a contiguous sequence of n tokens. Using n-grams as features recovers **local word order**: the bigram 'not good' tells a sentiment model what unigrams 'not' + 'good' cannot.

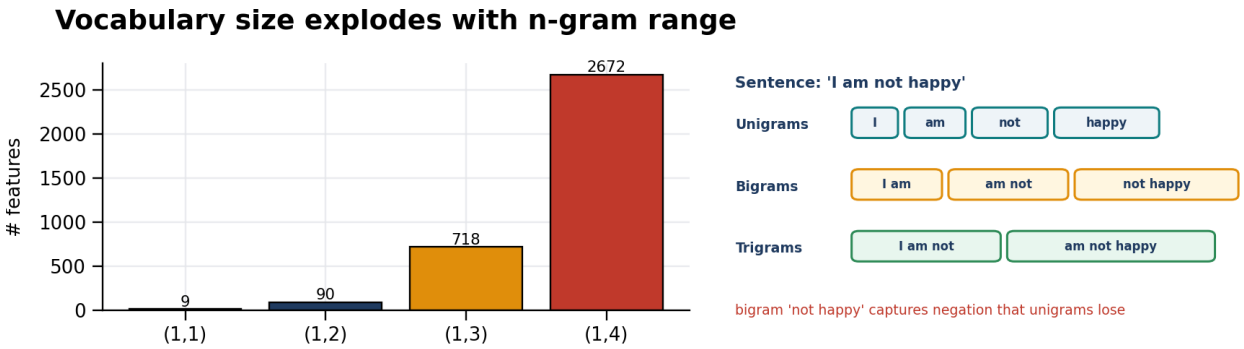


Figure: Left: feature count grows quickly with n (small synthetic corpus). Right: n-grams of one sentence.

Name	n	Features of 'the food was good'
Unigrams	1	the, food, was, good
Bigrams	2	the food, food was, was good
Trigrams	3	the food was, food was good

A sentence with N tokens has N − n + 1 n-grams. In sklearn, **ngram_range=(1, 2)** means unigrams + bigrams. Character n-grams (analyzer='char_wb', ngram_range=(2, 5)) are robust to typos and morphology.

ngrams.py

```
cv2 = CountVectorizer(ngram_range=(1, 2))
cv2.fit(docs).get_feature_names_out()
# ['bad' 'bad service' 'food' 'food good' 'food was' 'good' 'good food' 'good service' 'service' 'the' 'the food' 'was'
        'was bad' 'was good']
CountVectorizer(analyzer="char_wb", ngram_range=(3, 3)).fit(["good"]).get_feature_names_out()   # [' go' 'goo' 'od ' 'ood']
```

Interview tip: Trade-off: higher n captures more context but the vocabulary explodes and each feature becomes rarer (sparsity → overfitting). In practice (1,2) or (1,3) with min_df to drop rare n-grams.

5.5 TF-IDF (Term Frequency – Inverse Document Frequency)

BoW gives frequent-everywhere words like 'the' the same importance as distinctive words. TF-IDF **re-weights** counts: a term is important if it is frequent **in this document** (TF) but **rare across the corpus** (IDF).

$$\mathrm{TF}(t,d) = \frac{\mathrm{count}(t,d)}{\text{number of terms in } d} \qquad \mathrm{IDF}(t) = \log\frac{N}{\mathrm{df}(t)} \qquad \mathrm{TFIDF}(t,d) = \mathrm{TF}(t,d)\cdot\mathrm{IDF}(t)$$

textbook definition; N = number of documents, df(t) = number of documents containing t

Worked example (textbook formula, natural log)

N = 4. Document frequencies: the = 2, food = 3, was = 2, good = 2, bad = 2, service = 2.

Term	df	IDF = ln(4/df)	TF in D1 ('the food was good')	TF-IDF in D1
the	2	0.693	1/4 = 0.25	0.173
food	3	0.288	0.25	0.072
was	2	0.693	0.25	0.173
good	2	0.693	0.25	0.173

Term	df	IDF = ln(4/df)	TF in D1 ('the food was good')	TF-IDF in D1
bad	2	0.693	0	0
service	2	0.693	0	0

'food' appears in 3 of 4 documents so it gets the lowest weight; a word in every document would get IDF = ln(1) = 0.

What scikit-learn actually computes

$$\text{idf}(t) = \ln\frac{1+N}{1+\text{df}(t)} + 1 \qquad (\text{smooth_idf} = \text{True}), \qquad v_d = \frac{\text{tf}\cdot\text{idf}}{\|\text{tf}\cdot\text{idf}\|_2}$$

tf = raw count; +1 smoothing avoids division by zero and keeps idf ≥ 1; each row is L2-normalised

Doc	bad	food	good	service	the	was
D1	0.000	0.423	0.523	0.000	0.523	0.523
D2	0.523	0.423	0.000	0.000	0.523	0.523
D3	0.000	0.340	0.841	0.421	0.000	0.000
D4	0.707	0.000	0.000	0.707	0.000	0.000

Check D1: idf(food) = ln(5/4) + 1 = 1.223; idf(the) = idf(was) = idf(good) = ln(5/3) + 1 = 1.511. The vector [1.223, 1.511, 1.511, 1.511] has L2 norm ≈ 2.889, giving 0.423 and 0.523 — exactly the values above. In D3, 'good' (count 2) gets the largest weight 0.841.

tfidf.py

```
from sklearn.feature_extraction.text import TfidfVectorizer
tv = TfidfVectorizer(
    lowercase=True, stop_words=None,       # or "english" / custom list
    ngram_range=(1, 2),                    # unigrams + bigrams
    min_df=2, max_df=0.95,                 # drop very rare and near-ubiquitous terms (int = count, float = proportion)
    max_features=50000,                    # keep top features by frequency
    sublinear_tf=True,                     # tf → 1 + log(tf): dampens repeated words
    smooth_idf=True, norm="l2")
X_train = tv.fit_transform(train_texts)    # FIT on training data only
X_test = tv.transform(test_texts)          # reuse learned vocabulary + idf
dict(zip(tv.get_feature_names_out(), tv.idf_))
```

TF variant	Formula
raw count	f(t,d)
term frequency	f / |d|
binary	1 if present
log / sublinear	1 + log f
augmented	0.5 + 0.5·f / max f

Advantages	Limitations
down-weights common words automatically; better than BoW for classification & search; still sparse, fast and interpretable; the basis of BM25 search	still no semantics (synonyms 'car'/'automobile' unrelated); no word order beyond n-grams; high-dimensional sparse; OOV words ignored; vocabulary fixed at fit time

5.6 Other sparse techniques

Technique	Idea
HashingVectorizer	hash tokens into a fixed number of columns — no vocabulary stored, works for streaming/huge data; collisions; not invertible
BM25	ranking function = TF-IDF with term-frequency saturation (k_1) and document-length normalisation (b); used in search engines and hybrid RAG
Feature selection	chi^2 (SelectKBest(chi2, k)), mutual information — keep the most class-discriminative n-grams

5.7 Comparison of classical representations

	One-hot	BoW	N-gram BoW	TF-IDF
Unit	word	doc	doc	doc
Word order	kept (sequence)	lost	local order	lost (local with n-grams)
Weights	0/1	counts	counts	importance-weighted
Semantics	none	none	little	none
Dimension	V per token	V	V grows fast	V
Typical model	RNN input (legacy)	Naive Bayes	NB / LogReg	LogReg / SVM, search

CHAPTER 6

Statistical Language Models

What a language model is • chain rule • Markov assumption • n-gram LMs • MLE • sparsity & smoothing (Laplace, add-k, backoff, interpolation, Kneser-Ney) • perplexity • text generation

6.1 Language models

A **language model (LM)** assigns a probability to a sequence of words, or equivalently predicts the next word. It powers autocomplete, spelling correction, speech recognition, machine translation — and LLMs are simply very large neural language models.

$$P(w_1, \ldots, w_n) = \prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1})$$

chain rule of probability — exact but impossible to estimate for long histories

$$\text{Markov (n-gram) assumption}: \; P(w_i \mid w_{1:i-1}) \approx P(w_i \mid w_{i-n+1:i-1}), \qquad \text{bigram}: \; P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}w_i)}{C(w_{i-1})}$$

Example (corpus: '<s> I like tea </s>', '<s> I like coffee </s>', '<s> you like tea </s>'): P(I | <s>) = 2/3, P(like | I) = 2/2 = 1, P(tea | like) = 2/3. So P('<s> I like tea </s>') = 2/3 · 1 · 2/3 · 1 = 0.444.

6.2 Sparsity and smoothing

Any n-gram never seen in training gets probability 0, making the whole sentence probability 0. Smoothing moves some probability mass to unseen events:

$$\text{Laplace (add-1)}: \; P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + V} \qquad \text{add-k}: \; \frac{C + k}{C(w_{i-1}) + kV}$$

Method	Idea
Laplace / add-k	add pseudo-counts (simple; over-smooths)
Backoff (Katz, 'stupid backoff')	if trigram unseen, use bigram, then unigram
Interpolation	λ_3P(trigram) + λ_2P(bigram) + λ_1P(unigram), $\Sigma\lambda = 1$
Kneser-Ney	absolute discounting + 'continuation probability' (how many different contexts a word appears in) — best classical method; 'Francisco' is frequent but only after 'San'

6.3 Evaluating language models: perplexity

$$\text{PP}(W) = P(w_1 \ldots w_N)^{-1/N} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i \mid w_{<i})\right)$$

Perplexity is the effective 'branching factor' — how many words the model is choosing between on average. **Lower is better.** It is the exponential of the cross-entropy loss used to train neural LMs. Only compare perplexities computed with the same vocabulary/tokeniser.

6.4 Generating text with an n-gram model

bigram_lm.py

```
from collections import defaultdict, Counter
import random
def train_bigram(sentences):
    counts = defaultdict(Counter)
    for s in sentences:
        toks = ["<s>"] + s.lower().split() + ["</s>"]
        for a, b in zip(toks, toks[1:]): counts[a][b] += 1
    return counts
def generate(counts, max_len=15):
    w, out = "<s>", []
    while len(out) < max_len:
        nxt = counts[w]; w = random.choices(list(nxt), weights=nxt.values())[0]
        if w == "</s>": break
        out.append(w)
    return " ".join(out)
lm = train_bigram(["I like tea", "I like coffee", "you like tea"]); print(generate(lm))   # e.g. 'you like coffee'
# NLTK: from nltk.lm import MLE, Laplace, KneserNeyInterpolated; from nltk.lm.preprocessing import padded_everygram_pipeline
```

Limits of n-gram LMs: cannot capture long-range dependencies, huge count tables, sparse. Neural LMs (RNN → Transformer) solved these with dense embeddings and learned context — Chapters 10–11.

CHAPTER 7

Word Embeddings: Word2Vec (CBOW & Skip-gram), Avg Word2Vec, GloVe, FastText, Doc2Vec

Distributional hypothesis • dense vs sparse • Word2Vec architecture • CBOW • Skip-gram • softmax • negative sampling • hierarchical softmax • subsampling • training with Gensim • pre-trained vectors • analogies • Avg Word2Vec • TF-IDF-weighted Word2Vec • GloVe • FastText • Doc2Vec • evaluation & limitations

7.1 From sparse to dense: the idea of embeddings

Distributional hypothesis (Firth, 1957): "*You shall know a word by the company it keeps.*" Words that occur in similar contexts have similar meanings. An **embedding** is a dense, low-dimensional (50–300) real-valued vector learned so that words used in similar contexts get similar vectors.

	One-hot / BoW / TF-IDF	Word embeddings
Dimension	V (10k–1M)	50–300
Values	mostly 0 (sparse)	dense real numbers
Similarity	none (orthogonal)	cosine similarity reflects meaning
Learned?	counted	learned from a corpus (prediction task)
Generalises	no	yes — 'great' ≈ 'excellent' even if one is unseen in labelled data

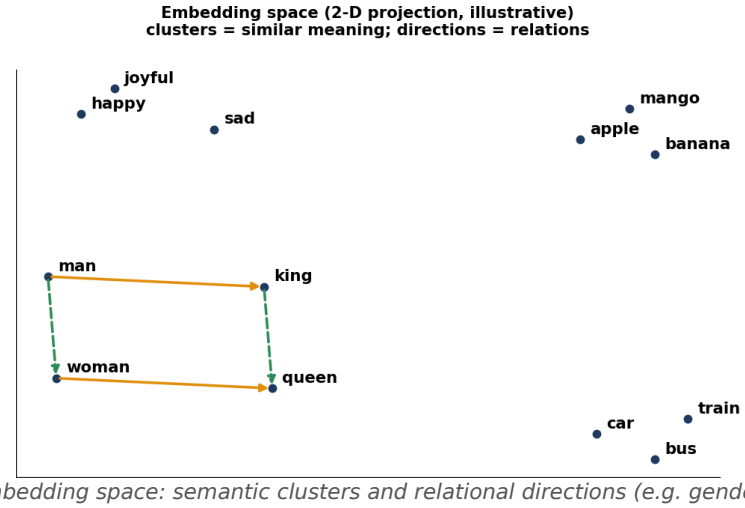


Figure: Embedding space: semantic clusters and relational directions (e.g. gender, royalty).

$$\vec{v}_{king} - \vec{v}_{man} + \vec{v}_{woman} \approx \vec{v}_{queen} \qquad \cos(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \, \|\vec{b}\|}$$

7.2 Word2Vec (Mikolov et al., Google, 2013)

Word2Vec is a shallow (one hidden layer, no activation) neural network trained on a **fake prediction task** using a sliding window over raw text. We don't care about the predictions themselves — the learned weight matrix **W** becomes the word embeddings. Two architectures: **CBOW** and **Skip-gram**.

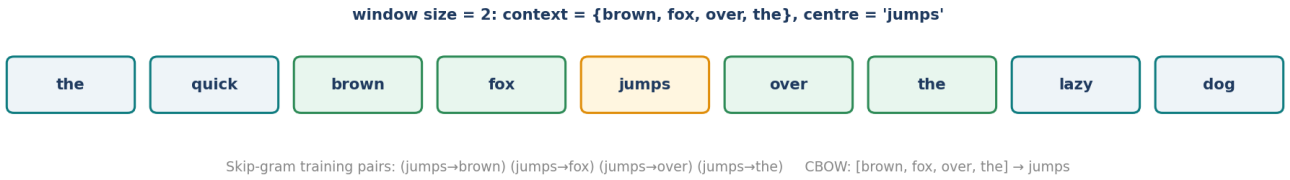


Figure: A sliding context window creates training examples automatically — self-supervised learning, no labels needed.

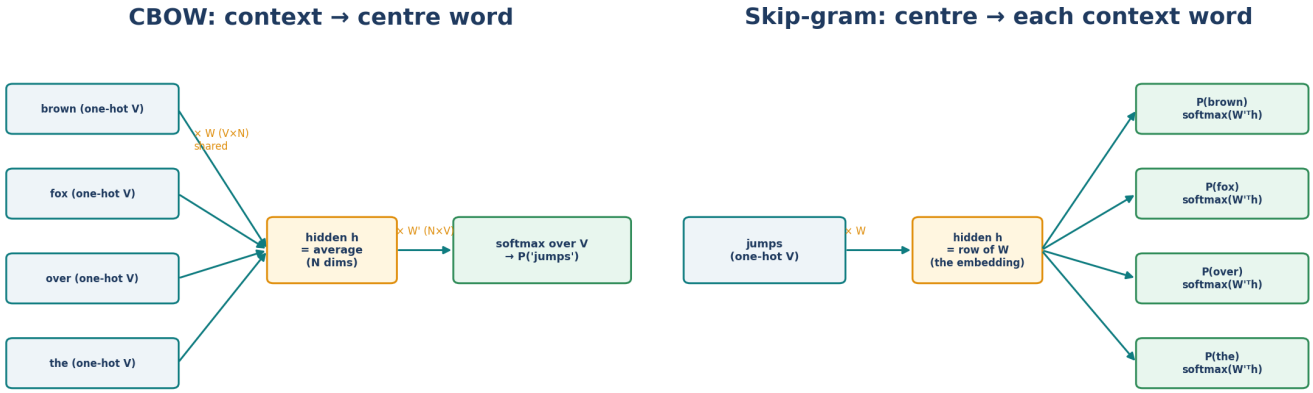


Figure: CBOW averages context vectors to predict the centre word; Skip-gram uses the centre word to predict each context word.

The architecture and the 'lookup' trick

Input: one-hot vector $x \in \mathbb{R}^V$. Hidden layer: $h = W^T x$ with $W \in \mathbb{R}^{V \times N}$ (N = embedding size). Because x is one-hot, h is simply the row of W for that word — an **embedding lookup**. Output: scores $u = W'^T h \in \mathbb{R}^V$ followed by softmax.

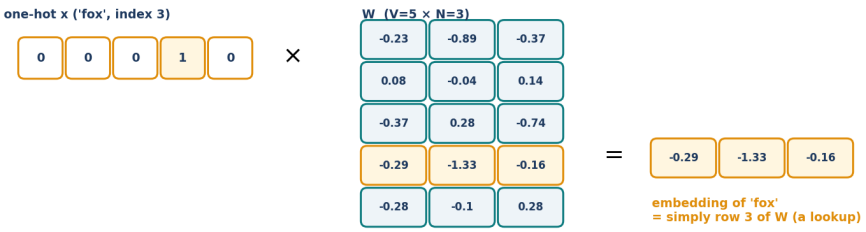


Figure: Multiplying a one-hot vector by W selects one row: the word's embedding.

CBOW (Continuous Bag of Words)

$$h = \frac{1}{2m} \sum_{-m \le j \le m,\ j \neq 0} W^T x_{t+j} \qquad P(w_t \mid \text{context}) = \frac{\exp({u'_{w_t}}^T h)}{\sum_{w=1}^{V} \exp({u'_w}^T h)} \qquad \mathcal{L} = -\log P(w_t \mid \text{context})$$

Predicts the **centre word** from the average of its 2m context word vectors. Faster to train (one prediction per window), smooths over context, works well for frequent words.

Skip-gram

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-m \le j \le m,\ j \neq 0} \log P(w_{t+j} \mid w_t), \qquad P(w_O \mid w_I) = \frac{\exp({v'_{w_O}}^{T} v_{w_I})}{\sum_{w=1}^{V} \exp({v'_{w}}^{T} v_{w_I})}$$

Predicts **each context word** from the centre word — 2m training pairs per position. Slower but gives better vectors for **rare words** and works well with small corpora.

	CBOW	Skip-gram
Predicts	centre from context	context from centre
Training pairs per window	1	2m
Speed	faster	slower
Rare words	weaker (averaged away)	better
Corpus size	large corpora	small–medium corpora
Gensim flag	sg=0 (default)	sg=1

Making training feasible: negative sampling & hierarchical softmax

The softmax denominator sums over the whole vocabulary (100k+ words) for every training pair — far too expensive.

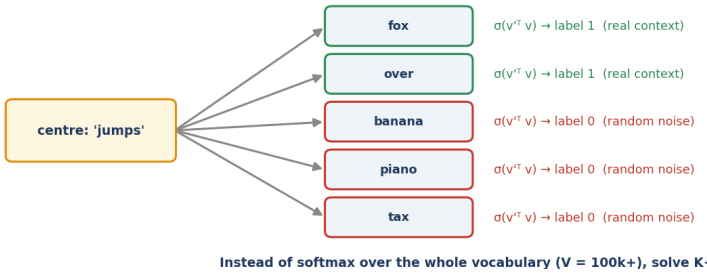


Figure: Negative sampling: tell real (centre, context) pairs apart from K random 'noise' pairs.

$$\mathcal{L}_{NEG} = -\log \sigma({v'_{w_O}}^{T} v_{w_I}) - \sum_{k=1}^{K} \log \sigma(-{v'_{w_k}}^{T} v_{w_I}), \qquad w_k \sim P_n(w) \propto f(w)^{3/4}$$

- **Negative sampling**: update only the positive word and K negatives (K = 5–20 for small data, 2–5 for large). Noise distribution = unigram frequency raised to 3/4 (gives rare words a bit more chance).
- **Hierarchical softmax**: arrange the vocabulary as a Huffman binary tree; P(word) = product of sigmoid decisions along its path → O(log V) instead of O(V).
- **Subsampling frequent words**: randomly drop very frequent words ('the') with probability 1 − √(t / f(w)) — faster training and better rare-word vectors.
- **Dynamic window**: sample the actual window size from 1..m so nearer words get more weight.

Training Word2Vec with Gensim

word2vec_gensim.py

```
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess
sentences = [simple_preprocess(t) for t in df["text"]]        # list of token lists
model = Word2Vec(
    sentences, vector_size=100,     # embedding dimension N
    window=5,                       # context size m
    min_count=2,                    # ignore rare words
    sg=1,                           # 1 = skip-gram, 0 = CBOW
    negative=5, hs=0,               # negative sampling (hs=1 → hierarchical softmax)
    sample=1e-3,                    # subsampling threshold
    epochs=10, workers=4, seed=42)
model.wv["loan"]                                    # 100-d vector
model.wv.most_similar("loan", topn=5)               # nearest neighbours by cosine
model.wv.similarity("loan", "emi")
model.wv.most_similar(positive=["king", "woman"], negative=["man"])   # analogy
model.save("w2v.model"); Word2Vec.load("w2v.model")
"pizza" in model.wv.key_to_index                     # OOV check

import gensim.downloader as api
glove = api.load("glove-wiki-gigaword-100")        # pre-trained (also 'word2vec-google-news-300', 3.6M words ×
        300)
```

Hyper-parameter	Effect
vector_size	100–300 typical; larger = more capacity, more data needed
window	small (2–5) → syntactic similarity (similar POS); large (5–10) → topical similarity
min_count	removes rare/noisy words; too high loses domain terms
sg / negative / hs / sample	algorithm choices above
epochs	more for small corpora

Pre-trained vs training your own

Use **pre-trained** vectors (Google News Word2Vec, GloVe, FastText Common Crawl) when your data is small and general-domain. **Train your own** (or continue training) when the domain has special vocabulary (banking: 'NEFT', 'EMI', 'KYC') and you have enough text (millions of tokens).

7.3 Average Word2Vec — from word vectors to document vectors

Classifiers need one fixed-length vector per document. The simplest way: **average the vectors of all in-vocabulary words** in the document.

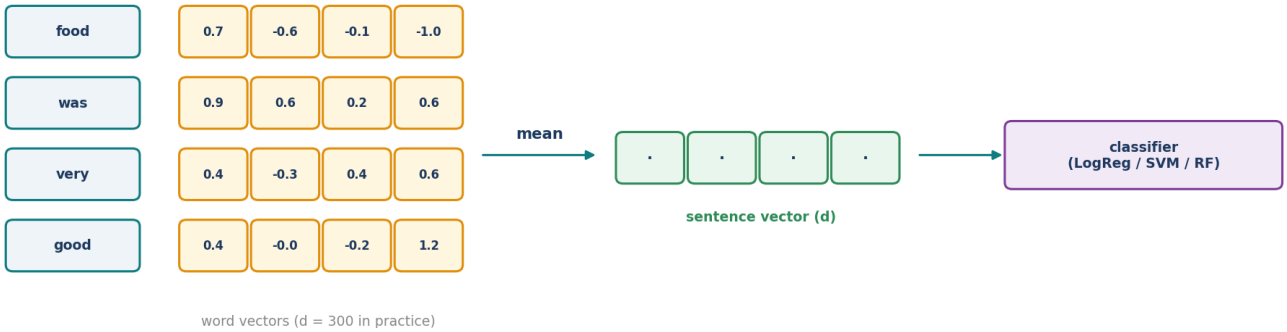


Figure: Average Word2Vec: each word → vector; mean over words → document vector → classifier.

$$\vec{d} = \frac{1}{|d'|} \sum_{w \in d'} \vec{v}_w, \qquad d' = \{w \in d : \ w \in \text{vocab}\}$$

avg_word2vec.py

```
import numpy as np
def avg_word2vec(tokens, wv, dim=100):
    vecs = [wv[w] for w in tokens if w in wv.key_to_index]
    return np.mean(vecs, axis=0) if vecs else np.zeros(dim)          # all-OOV document → zero vector
X = np.vstack([avg_word2vec(simple_preprocess(t), model.wv) for t in df["text"]])   # (n_docs, 100)
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(max_iter=1000).fit(X_train, y_train)
```

TF-IDF weighted Word2Vec

$$\vec{d} = \frac{\sum_{w \in d'} \text{tfidf}(w, d)\, \vec{v}_w}{\sum_{w \in d'} \text{tfidf}(w, d)}$$

important words count more; stop words contribute little

tfidf_word2vec.py

```
tv = TfidfVectorizer(tokenizer=str.split, lowercase=False).fit(train_clean)
idf = dict(zip(tv.get_feature_names_out(), tv.idf_))
def tfidf_w2v(tokens, wv, dim=100):
    num, den = np.zeros(dim), 0.0
    for w, c in Counter(tokens).items():
        if w in wv.key_to_index and w in idf:
            weight = c * idf[w]; num += weight * wv[w]; den += weight
    return num / den if den else num
```

Pros of averaging	Cons
dense, low-dimensional (100–300) features; uses semantic similarity; works with any classifier; handles synonyms unseen in labelled data	loses word order and negation ('not good' ≈ 'good not'); long documents get 'washed out'; OOV words ignored; often slightly below TF-IDF on topic-heavy tasks — try both

7.4 GloVe (Global Vectors, Stanford 2014)

$$J = \sum_{i,j=1}^{V} f(X_{ij})\left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij}\right)^2, \qquad f(x) = \min\left(1, (x/x_{max})^{3/4}\right)$$

GloVe builds a global word–word **co-occurrence matrix** X (how often word j appears in the context of word i) and learns vectors whose dot products approximate log co-occurrence counts. Ratios of co-occurrence probabilities encode meaning (P(solid|ice)/P(solid|steam) is large). Word2Vec = local, predictive; GloVe = global, count-based; both give similar quality.

7.5 FastText (Facebook AI, 2016)

Represents each word as the **sum of its character n-gram vectors** (n = 3–6) plus the word itself: 'where' → <wh, whe, her, ere, re>, <where>. Therefore it can build vectors for **OOV and misspelled words** ('whereee') and captures morphology — great for morphologically rich languages (Hindi, Tamil) and noisy user text. FastText is also a very fast linear text classifier.

fasttext.py

```
from gensim.models import FastText
ft = FastText(sentences, vector_size=100, window=5, min_count=2, sg=1, min_n=3, max_n=6, epochs=10)
ft.wv["loanss"]          # works even though 'loanss' was never seen
```

7.6 Doc2Vec (Paragraph Vectors)

Learns a vector for each **document** jointly with word vectors. **PV-DM** (like CBOW: paragraph vector + context words → next word) and **PV-DBOW** (like skip-gram: paragraph vector → words in the paragraph). New documents need inference (gradient steps). Modern alternative: sentence-transformers.

doc2vec.py

```
from gensim.models.doc2vec import Doc2Vec, TaggedDocument
tagged = [TaggedDocument(simple_preprocess(t), [i]) for i, t in enumerate(texts)]
d2v = Doc2Vec(tagged, vector_size=100, window=5, min_count=2, epochs=20, dm=1)
vec = d2v.infer_vector(simple_preprocess("loan emi not deducted"))
```

7.7 Evaluating and visualising embeddings

- **Intrinsic**: word similarity benchmarks (WordSim-353, SimLex-999 — correlation with human scores), analogy tasks (Google analogy set accuracy).
- **Extrinsic**: improvement on a downstream task (classification F1, NER F1) — what really matters.
- **Visualise**: reduce to 2-D with PCA or t-SNE and plot; check that clusters make sense.
- Check **bias**: embeddings learn societal stereotypes ('doctor' closer to 'man'); test and mitigate.

7.8 Limitations of static embeddings

One vector per word regardless of context: 'bank' in 'river bank' and 'bank loan' share a vector (**polysemy** problem). Antonyms can be close ('good'/'bad' appear in the same contexts). No sentence-level meaning. These are solved by **contextual embeddings** — ELMo (BiLSTM LM), then BERT and other transformers (Chapter 11).

Model	Type	Unit	OOV	Context-aware
One-hot/BoW/TF-IDF	count	word/doc	no	no
Word2Vec	predictive, local window	word	no	no
GloVe	count, global matrix	word	no	no
FastText	predictive + subwords	char n-grams	yes	no
Doc2Vec	predictive	document	infer	no
ELMo	BiLSTM LM	word (chars)	yes	yes
BERT / sentence-transformers	transformer	subword / sentence	yes	yes

CHAPTER 8

Text Classification End-to-End

Problem types • the pipeline • feature options • Naive Bayes (math + worked example) • Logistic Regression • SVM • trees/boosting • imbalanced text • evaluation • error analysis • Project 1: spam detection • Project 2: sentiment analysis (lexicon vs ML) • comparing BoW / TF-IDF / Avg Word2Vec

8.1 What is text classification?

Assign one or more predefined labels to a piece of text. The most common NLP task in industry.

Type	Example	Output layer / setup
Binary	spam vs ham, toxic vs clean	sigmoid / one probability
Multi-class (one label)	ticket → billing / tech / account / other	softmax
Multi-label (many labels)	news article → {politics, economy}	one sigmoid per label; OneVsRest / MultiOutput; threshold each
Hierarchical	product → Electronics › Mobile › Android	cascade or flat with constraints
Ordinal	rating 1–5	regression or ordinal classification

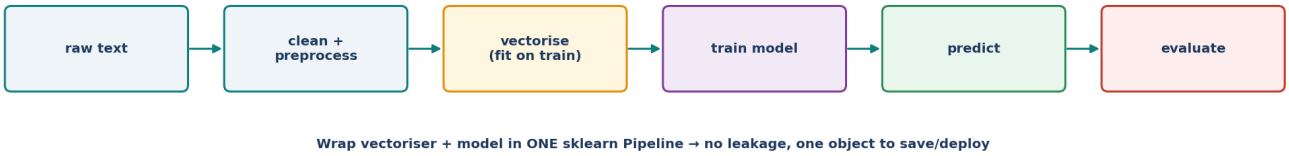


Figure: The text-classification pipeline.

8.2 Algorithms

Multinomial Naive Bayes

$$P(c \mid d) \propto P(c) \prod_{i=1}^{n} P(w_i \mid c) \qquad \hat{c} = \arg\max_c \left[\log P(c) + \sum_i \log P(w_i \mid c) \right]$$

$$P(w \mid c) = \frac{\text{count}(w, c) + \alpha}{\sum_{w'} \text{count}(w', c) + \alpha V} \qquad (\alpha = 1 : \text{ Laplace smoothing})$$

'Naive' = words are assumed conditionally independent given the class. Logs prevent numeric underflow from multiplying many small probabilities. Smoothing prevents an unseen word from zeroing the whole product.

Worked example

Training doc	Class
free money now	spam
win free prize	spam
meeting at noon	ham
project meeting notes	ham

Priors: P(spam) = P(ham) = 0.5. Vocabulary V = {free, money, now, win, prize, meeting, at, noon, project, notes} → 10 words. Spam has 6 tokens, ham has 6 tokens. Test: **'free meeting'**.

- P(free | spam) = (2+1)/(6+10) = 3/16; P(meeting | spam) = (0+1)/16 = 1/16 → score(spam) = 0.5 · 3/16 · 1/16 = 0.00586
- P(free | ham) = (0+1)/16 = 1/16; P(meeting | ham) = (2+1)/16 = 3/16 → score(ham) = 0.5 · 1/16 · 3/16 = 0.00586
- Tie! Add one more word: 'free money meeting' → spam gets × P(money|spam) = 2/16, ham gets × 1/16 → **spam wins** (2× more likely).

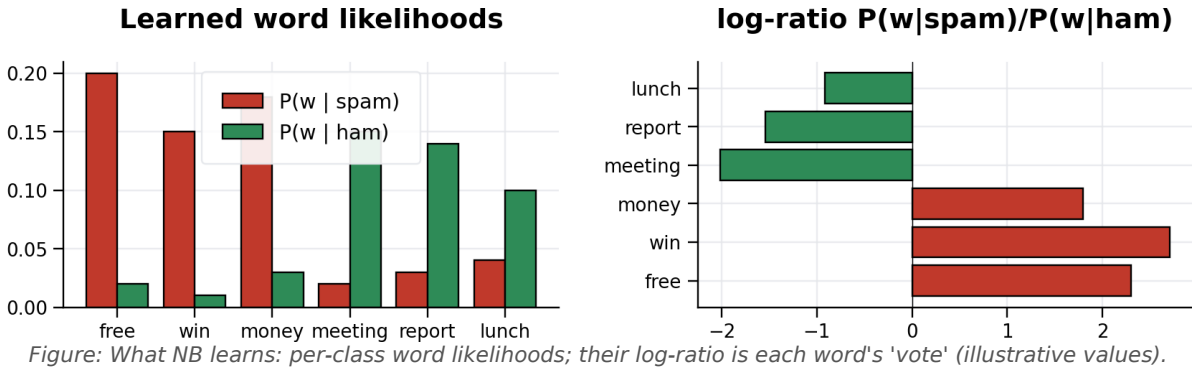


Figure: What NB learns: per-class word likelihoods; their log-ratio is each word's 'vote' (illustrative values).

NB variant	Features
MultinomialNB	counts / TF-IDF (text default)
BernoulliNB	binary presence; penalises absent words
ComplementNB	better for imbalanced text
GaussianNB	continuous features (e.g. averaged embeddings)

Logistic Regression, SVM and others

Model	Why it works on text	Tips
Logistic Regression	linear model on sparse TF-IDF; calibrated probabilities; coefficients = interpretable word importances	C (inverse regularisation), class_weight='balanced', solver='liblinear'/'saga'
Linear SVM	max-margin; excellent in high-dimensional sparse spaces	LinearSVC; no probabilities (use CalibratedClassifierCV)
Random Forest / XGBoost	non-linear; good on dense features (Avg Word2Vec, embeddings, meta-features)	slow on huge sparse TF-IDF
kNN	uses similarity; good with embeddings	slow at scale
Deep learning / transformers	learn context, word order	Chapters 10–11

8.3 Project 1 — SMS spam detection (end-to-end)

spam_classifier.py

```
import pandas as pd, joblib
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

df = pd.read_csv("spam.csv", encoding="latin-1")[["v1", "v2"]].rename(columns={"v1": "label", "v2": "text"})
df = df.drop_duplicates("text"); df["y"] = (df["label"] == "spam").astype(int)
print(df["y"].value_counts(normalize=True))                  # ~13% spam → imbalanced
df["clean"] = df["text"].apply(lambda t: preprocess(t, remove_stop=False))   # function from Chapter 3

X_tr, X_te, y_tr, y_te = train_test_split(df["clean"], df["y"], test_size=0.2, stratify=df["y"], random_state=42)

pipe = Pipeline([("vec", TfidfVectorizer()), ("clf", MultinomialNB())])
grid = {"vec": [CountVectorizer(), TfidfVectorizer(sublinear_tf=True)], "vec__ngram_range": [(1, 1), (1, 2)],
        "vec__min_df": [1, 2],
        "clf": [MultinomialNB(), LogisticRegression(max_iter=2000, class_weight="balanced")]}
search = GridSearchCV(pipe, grid, scoring="f1", cv=StratifiedKFold(5, shuffle=True, random_state=42), n_jobs=-1)
search.fit(X_tr, y_tr); print(search.best_params_, search.best_score_)

best = search.best_estimator_
print(classification_report(y_te, best.predict(X_te), target_names=["ham", "spam"]))
print(confusion_matrix(y_te, best.predict(X_te)))
joblib.dump(best, "spam_pipeline.joblib")                    # cleaning function + this pipeline = deployable model
```

Interpreting the model

interpret.py

```
vec, clf = best.named_steps["vec"], best.named_steps["clf"]
names = vec.get_feature_names_out()
coef = clf.coef_[0] if hasattr(clf, "coef_") else clf.feature_log_prob_[1] - clf.feature_log_prob_[0]
top = coef.argsort()
print("spammy:", names[top[-15:]]); print("hammy:", names[top[:15]])
```

8.4 Project 2 — Sentiment analysis

Approach A: lexicon-based (no training)

Sum polarity scores from a sentiment dictionary with rules for negation, intensifiers ('very'), punctuation and emojis. **VADER** is tuned for social media; **TextBlob** gives polarity in [−1, 1] and subjectivity. Fast and explainable but weak on domain language and sarcasm.

vader.py

```
from nltk.sentiment import SentimentIntensityAnalyzer    # nltk.download("vader_lexicon")
sia = SentimentIntensityAnalyzer()
sia.polarity_scores("The app is NOT good at all!! 😡")   # {'neg':..., 'neu':..., 'pos':..., 'compound':
        -0.6...}
label = "pos" if s["compound"] >= 0.05 else "neg" if s["compound"] <= -0.05 else "neu"
```

Approach B: machine learning with three feature sets

sentiment_compare.py

```
from sklearn.svm import LinearSVC
from sklearn.model_selection import cross_val_score
feats = {
  "BoW (1-gram)":       Pipeline([("v", CountVectorizer()), ("m", LogisticRegression(max_iter=2000))]),
  "TF-IDF (1,2)-gram":  Pipeline([("v", TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True, min_df=2)), ("m",
        LinearSVC())]),
}
for name, p in feats.items(): print(name, cross_val_score(p, texts, y, cv=5, scoring="f1_macro").mean())

X_w2v = np.vstack([avg_word2vec(simple_preprocess(t), wv, 300) for t in texts])   # pre-trained 300-d vectors
print("Avg Word2Vec", cross_val_score(LogisticRegression(max_iter=2000), X_w2v, y, cv=5, scoring="f1_macro").mean())
```

Typical findings to discuss in interviews: TF-IDF with bigrams is a very strong baseline for sentiment (bigrams capture negation); Avg Word2Vec helps when labelled data is small and vocabulary varies, but loses negation; fine-tuned transformers usually win when you can afford them; always compare on the same CV folds.

Approach C: aspect-based sentiment

'Camera is great but battery is poor' → camera: positive, battery: negative. Extract aspects (noun phrases / dependency relations like amod) and classify sentiment per aspect, or prompt an LLM for JSON {aspect: sentiment}.

8.5 Handling real-world issues

Issue	Fixes
Class imbalance	stratified splits; class_weight='balanced'; ComplementNB; resample (SMOTE works on dense features); tune threshold; report macro-F1 / PR-AUC
Small data	pre-trained embeddings, data augmentation, transfer learning, LLM few-shot
Noisy labels	label review, confident learning (cleanlab), agreement between annotators (Cohen's κ)
Long documents	truncate head+tail, chunk and aggregate, hierarchical models
Code-mixed / multilingual	char n-grams, FastText, multilingual transformers (mBERT, XLM-R, MuRIL)
Data leakage	fit vectoriser inside Pipeline; deduplicate before splitting
Concept drift	new slang/products → monitor OOV rate and retrain

8.6 Evaluation metrics for classification

$$\text{Precision} = \frac{TP}{TP+FP} \quad \text{Recall} = \frac{TP}{TP+FN} \quad F_1 = \frac{2PR}{P+R} \quad \text{Macro-F1} = \frac{1}{C}\sum_c F_1^{(c)}$$

Micro-average pools all decisions (equals accuracy for single-label multi-class); macro-average treats each class equally (exposes poor minority-class performance); weighted averages by support. For multi-label also use Hamming loss and subset accuracy.

8.7 Error analysis checklist

- Look at false positives and false negatives side by side.
- Group errors: negation, sarcasm, very short texts, OOV/typos, wrong labels, multiple topics.
- Check top features per class for leakage (e.g. a signature that reveals the label).
- Fix the biggest group first: more data for it, new features (bigrams, char n-grams), better preprocessing, or a stronger model.

CHAPTER 9

Text Similarity, Search, Topic Modelling & Clustering

Cosine, Jaccard, Euclidean, edit distance • semantic similarity • information retrieval & BM25 • LSA • LDA • NMF • BERTopic • document clustering • keyword extraction

9.1 Similarity measures

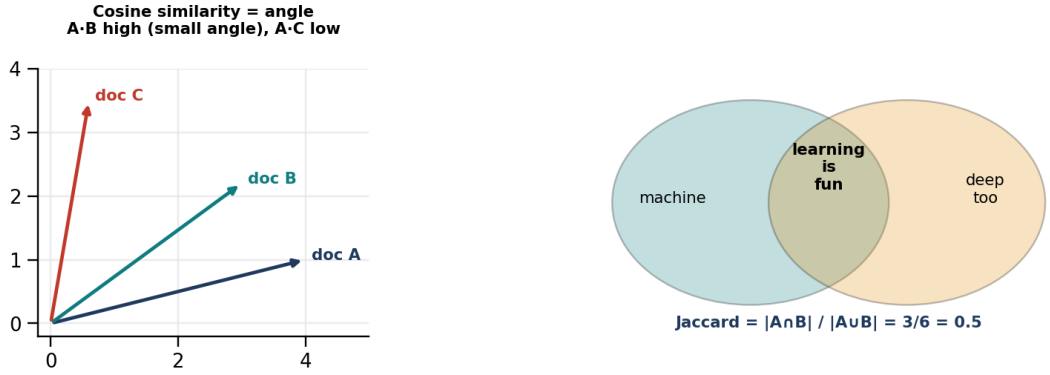


Figure: Cosine similarity compares directions; Jaccard compares token sets.

$$\cos(A,B)=\frac{A\cdot B}{\|A\|\|B\|} \qquad J(A,B)=\frac{|A\cap B|}{|A\cup B|} \qquad d_{Euclid}=\|A-B\|_2$$

Measure	On	Use
Cosine	TF-IDF / embedding vectors	document & semantic similarity (length-independent)
Jaccard	sets of tokens / shingles	near-duplicate detection (MinHash for scale)
Levenshtein (edit) distance	characters	spelling correction, fuzzy matching of names ('Bengaluru' vs 'Bangalore' → still needs a synonym list)
Euclidean / Manhattan	dense vectors	clustering (normalise first)
Word Mover's Distance	word embeddings	semantic doc distance (slow)

$$D[i,j]=\min\big(D[i-1,j]+1,\ D[i,j-1]+1,\ D[i-1,j-1]+[a_i\neq b_j]\big)$$

deletion, insertion, substitution — dynamic programming, O(|a|·|b|); 'kitten' → 'sitting' = 3

similarity.py

```
from sklearn.metrics.pairwise import cosine_similarity
X = TfidfVectorizer().fit_transform(faq_questions)
q = tv.transform(["how do I reset my password"])
best = cosine_similarity(q, X).ravel().argsort()[::-1][:3]       # top-3 most similar FAQs

from sentence_transformers import SentenceTransformer, util
m = SentenceTransformer("all-MiniLM-L6-v2")
emb = m.encode(faq_questions, normalize_embeddings=True)
util.cos_sim(m.encode("forgot login credentials", normalize_embeddings=True), emb)   # semantic match without shared words
```

9.2 Information retrieval & BM25

$$\text{BM25}(q,d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\,\dfrac{|d|}{\text{avgdl}}\right)} \qquad k_1 \approx 1.2\text{–}2,\ b \approx 0.75$$

An inverted index maps each term to the documents containing it, so search touches only relevant documents. BM25 improves TF-IDF with saturation (the 10th occurrence of a word adds little) and length normalisation. Evaluation: precision@k, recall@k, MRR, nDCG. Modern search = BM25 + dense embeddings (hybrid) + re-ranking — the foundation of RAG.

9.3 Topic modelling

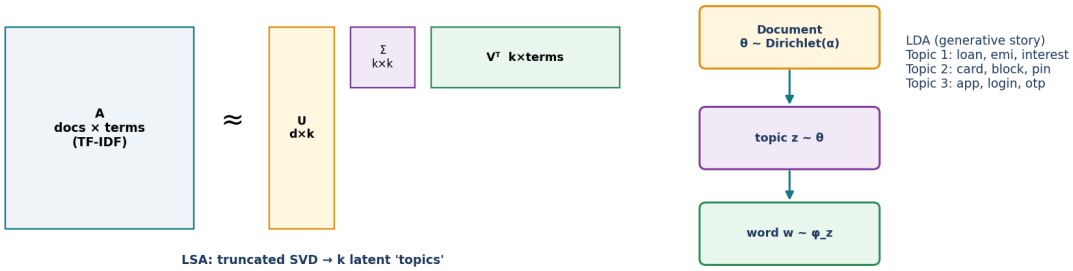


Figure: LSA factorises the document-term matrix; LDA is a generative probabilistic model of topics.

Method	Idea	Notes
LSA / LSI	truncated SVD of the TF-IDF matrix: A ≈ U_k Σ_k V_kᵀ; latent dimensions = topics	fast, linear algebra; topics can have negative weights (hard to interpret)
LDA (Latent Dirichlet Allocation)	each document is a mixture of topics (θ ~ Dir(α)); each topic is a distribution over words (φ ~ Dir(β)); fit with Gibbs sampling / variational Bayes	interpretable; choose k by coherence score; use BoW counts
NMF	A ≈ WH with non-negative factors	interpretable parts-based topics, works well with TF-IDF
BERTopic	sentence embeddings → UMAP → HDBSCAN clusters → class-based TF-IDF keywords	modern, semantic, handles short texts

topics.py

```
from sklearn.decomposition import LatentDirichletAllocation, NMF, TruncatedSVD
cv = CountVectorizer(max_df=0.9, min_df=5, stop_words="english"); X = cv.fit_transform(tickets)
lda = LatentDirichletAllocation(n_components=8, random_state=0).fit(X)
words = cv.get_feature_names_out()
for k, topic in enumerate(lda.components_): print(k, [words[i] for i in topic.argsort()[-8:]])
doc_topics = lda.transform(X)                       # each row sums to 1
# gensim: LdaModel + CoherenceModel(coherence="c_v") to choose the number of topics
```

9.4 Document clustering and keyword extraction

- Cluster TF-IDF (normalised) or sentence embeddings with K-Means / HDBSCAN; label clusters by top TF-IDF terms; evaluate with silhouette score and manual inspection.
- Keyword extraction: top TF-IDF terms per doc, RAKE (co-occurrence of content words), TextRank (PageRank on a word graph), YAKE, KeyBERT (embedding similarity between doc and candidate phrases).

CHAPTER 10

Deep Learning for NLP

Keras tokenizer & sequences • padding & OOV • Embedding layer • RNN, LSTM, GRU, Bi-LSTM for text • 1-D CNN for text • seq2seq & attention • sequence labelling • character models

10.1 Preparing text for neural networks

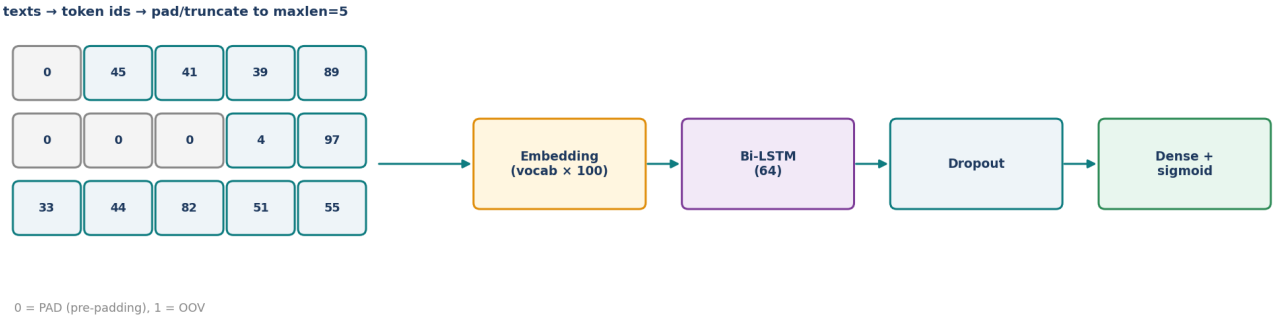


Figure: Text → integer ids → padded sequences → Embedding → recurrent layers → output.

text_to_sequences.py

```
import tensorflow as tf
from tensorflow.keras import layers
vectorize = layers.TextVectorization(max_tokens=20000, output_mode="int", output_sequence_length=100)   # 0 = PAD, 1 = OOV
        ([UNK])
vectorize.adapt(train_texts)                          # learns vocabulary from TRAINING text only
vectorize(["the food was not good"])                  # → [[ 2, 15, 9, 31, 22, 0, 0, ...]]
# classic API: Tokenizer(num_words=20000, oov_token="<OOV>").fit_on_texts(...); texts_to_sequences(...);
        pad_sequences(seqs, maxlen=100, padding="post", truncating="post")
```

Choose maxlen from the length distribution (e.g. 95th percentile). **Pre-padding** is often better for LSTMs (real tokens close to the output); **masking** (mask_zero=True) tells layers to ignore pads.

10.2 The Embedding layer

An Embedding layer is a trainable lookup table of shape (vocab_size × embedding_dim) — exactly the W matrix of Word2Vec. It can be initialised randomly (learned with the task) or with **pre-trained** Word2Vec/GloVe vectors, then frozen (trainable=False) or fine-tuned.

pretrained_embedding.py

```
emb_matrix = np.zeros((vocab_size, 300))
for i, w in enumerate(vectorize.get_vocabulary()):
    if w in glove: emb_matrix[i] = glove[w]
emb = layers.Embedding(vocab_size, 300, embeddings_initializer=tf.keras.initializers.Constant(emb_matrix), trainable=False,
        mask_zero=True)
```

10.3 RNN, LSTM, GRU and Bi-LSTM for text

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b) \qquad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad h_t = o_t \odot \tanh(C_t)$$

An RNN reads tokens one at a time and keeps a hidden state; simple RNNs suffer vanishing gradients, so LSTMs (forget/input/output gates + cell state) or GRUs (update/reset gates) are used. **Bidirectional** LSTMs read the sentence both ways — great for classification and tagging where the whole text is available. For classification use the last hidden state (return_sequences=False) or pool over all states; for tagging (NER/POS) use return_sequences=True and a per-token output.

bilstm_classifier.py

```
inputs = tf.keras.Input(shape=(1,), dtype=tf.string)
x = vectorize(inputs)
x = layers.Embedding(20000, 128, mask_zero=True)(x)
x = layers.Bidirectional(layers.LSTM(64, return_sequences=True))(x)
x = layers.GlobalMaxPooling1D()(x)                # or Bidirectional(LSTM(64)) without return_sequences
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)   # Dense(num_classes, softmax) for multi-class
model = tf.keras.Model(inputs, outputs)
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10, callbacks=[tf.keras.callbacks.EarlyStopping(patience=2,
        restore_best_weights=True)])
```

10.4 1-D CNN for text

Convolution filters of width k slide over the embedding sequence and act as learned **n-gram detectors** ('not good', 'highly recommend'); global max pooling keeps the strongest signal of each filter. Several widths (3, 4, 5) in parallel = Kim-CNN (2014). Fast, parallel, strong for short-text classification; weaker at long-range dependencies.

text_cnn.py

```
x = layers.Embedding(20000, 128)(vectorize(inputs))
convs = [layers.GlobalMaxPooling1D()(layers.Conv1D(100, k, activation="relu")(x)) for k in (3, 4, 5)]
x = layers.Dropout(0.5)(layers.Concatenate()(convs)); outputs = layers.Dense(1, activation="sigmoid")(x)
```

10.5 Sequence-to-sequence, attention and sequence labelling

- **Encoder–decoder (seq2seq)**: an encoder LSTM compresses the input; a decoder LSTM generates output tokens one by one (translation, summarisation, chatbots); training with teacher forcing; decoding with greedy or beam search.
- **Attention**: at each decoding step compute weights over all encoder states and use their weighted sum as context — removes the fixed-vector bottleneck and gives alignments. Led directly to the Transformer.
- **Sequence labelling** (POS, NER): BiLSTM produces a vector per token → softmax per token, or a CRF layer on top to enforce valid tag sequences (BiLSTM-CRF).
- **Character-level models**: char-CNN/LSTM handle typos and OOV; combined with word embeddings in classic NER systems.

CHAPTER 11

Transformers & Modern NLP

Self-attention • transformer architecture • subword tokenisation (BPE, WordPiece, SentencePiece) • BERT • GPT • T5 • Hugging Face pipelines • fine-tuning for classification & NER • sentence embeddings • LLMs for NLP tasks • when to use what

11.1 Self-attention in one page

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \qquad Q = XW^Q,\ K = XW^K,\ V = XW^V$$

Every token builds a query ('what am I looking for?'), a key ('what do I contain?') and a value ('what do I pass on?'). Each output vector is a weighted mix of all value vectors, with weights from query–key similarity — so every word's representation becomes **contextual**: 'bank' gets different vectors in 'river bank' and 'bank loan'. Multi-head attention runs several of these in parallel; positional encodings add word order; stacks of attention + feed-forward layers with residuals and layer norm form the Transformer (Vaswani et al., 2017).

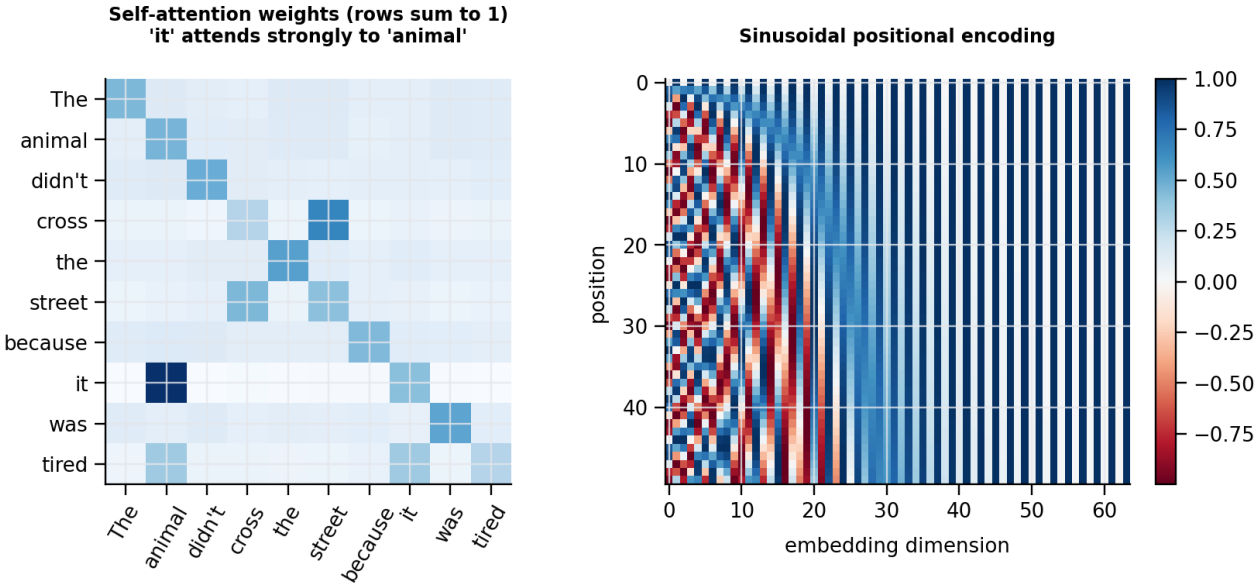


Figure: Attention weights (left) and positional encodings (right).

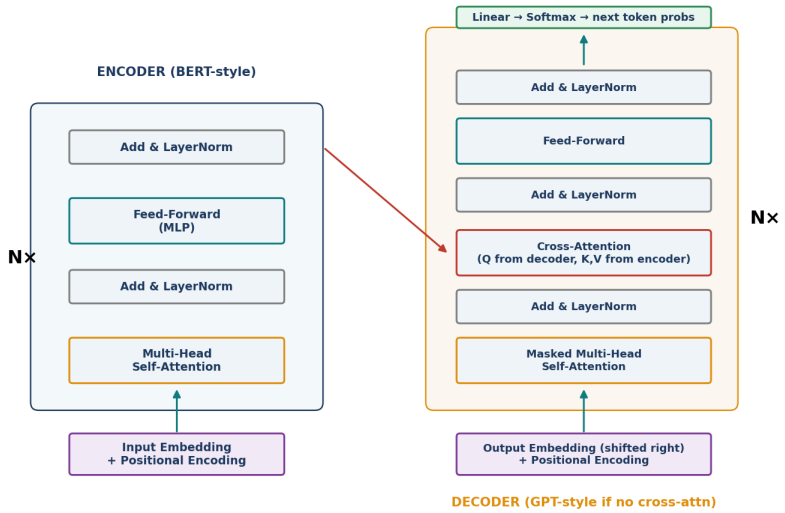


Figure: Transformer encoder (BERT-style) and decoder (GPT-style).

11.2 Subword tokenisation

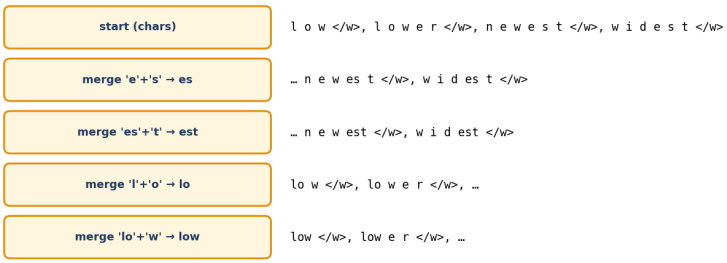


Figure: Byte-Pair Encoding learns merges of the most frequent adjacent symbol pairs.

Algorithm	How	Models
BPE	start from characters/bytes, repeatedly merge the most frequent pair until vocab size reached	GPT-2/3/4, RoBERTa, LLaMA 3
WordPiece	merge the pair that most increases training-data likelihood; continuation pieces marked ##	BERT, DistilBERT
Unigram LM (SentencePiece)	start big, prune pieces that least reduce likelihood; treats space as a symbol (▁); language-agnostic	T5, ALBERT, LLaMA 1/2, mT5

subword.py

```
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("bert-base-uncased")
tok.tokenize("Tokenization is unbelievable")      # ['token', '##ization', 'is', 'unbelievable']
enc = tok("Hello NLP", padding="max_length", truncation=True, max_length=8)
enc["input_ids"], enc["attention_mask"]           # [CLS]=101 ... [SEP]=102, pads=0 ; mask 1 for real tokens
```

11.3 The BERT, GPT and T5 families

	BERT (encoder)	GPT (decoder)	T5 / BART (encoder–decoder)
Attention	bidirectional	causal (left-to-right)	bidirectional encoder + causal decoder
Pre-training	Masked LM (15% tokens) + next sentence prediction	next-token prediction	span corruption / denoising
Best for	classification, NER, extractive QA, embeddings	generation, chat, few-shot	translation, summarisation, text-to-text
Variants	RoBERTa, DistilBERT, ALBERT, DeBERTa, ELECTRA, MuRIL, IndicBERT	GPT-2/3/4-class, LLaMA, Mistral, Gemma	Flan-T5, mT5, BART, Pegasus

11.4 Hugging Face pipelines (zero code training)

hf_pipelines.py

```
from transformers import pipeline
pipeline("sentiment-analysis")("I love this phone but the battery is weak")
pipeline("ner", aggregation_strategy="simple")("Mansavi works at LTIMindtree in Bengaluru")
pipeline("zero-shot-classification", model="facebook/bart-large-mnli")("My card got blocked", candidate_labels=["cards",
        "loans", "accounts"])
pipeline("question-answering")(question="Where is the office?", context="The office is in Bengaluru near MG Road.")
pipeline("summarization", model="facebook/bart-large-cnn")(long_text, max_length=120, min_length=30)
pipeline("translation_en_to_de", model="t5-small")("How are you?")
pipeline("fill-mask", model="bert-base-uncased")("The bank approved my [MASK].")
pipeline("text-generation", model="gpt2")("Once upon a time", max_new_tokens=30)
```

11.5 Fine-tuning BERT for text classification

finetune_bert.py

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification, TrainingArguments, Trainer,
        DataCollatorWithPadding
import numpy as np, evaluate
ds = load_dataset("csv", data_files={"train": "train.csv", "test": "test.csv"})          # columns: text, label
ckpt = "distilbert-base-uncased"; tok = AutoTokenizer.from_pretrained(ckpt)
ds = ds.map(lambda b: tok(b["text"], truncation=True, max_length=256), batched=True)
model = AutoModelForSequenceClassification.from_pretrained(ckpt, num_labels=3, id2label={0: "neg", 1: "neu", 2: "pos"},
        label2id={"neg": 0, "neu": 1, "pos": 2})
f1 = evaluate.load("f1")
def metrics(p): return f1.compute(predictions=np.argmax(p.predictions, -1), references=p.label_ids, average="macro")
args = TrainingArguments("out", learning_rate=2e-5, per_device_train_batch_size=16, num_train_epochs=3, weight_decay=0.01,
                         eval_strategy="epoch", save_strategy="epoch", load_best_model_at_end=True,
        metric_for_best_model="f1")
trainer = Trainer(model=model, args=args, train_dataset=ds["train"], eval_dataset=ds["test"],
        data_collator=DataCollatorWithPadding(tok), compute_metrics=metrics)
trainer.train(); trainer.save_model("sentiment-distilbert")
```

Tips: small learning rates (2e-5 – 5e-5), 2–4 epochs, warm-up, dynamic padding, mixed precision; for NER use AutoModelForTokenClassification and align word labels to sub-word tokens (label only the first sub-token, others −100).

11.6 Sentence embeddings and LLMs for NLP tasks

- **Sentence-transformers** (SBERT) fine-tune BERT in a siamese setup with contrastive loss → one vector per sentence for semantic search, clustering, deduplication, RAG.
- **LLMs** (GPT, Gemini, Claude, LLaMA) solve many NLP tasks by prompting: zero/few-shot classification, extraction to JSON, summarisation, translation, rewriting. Use structured output, low temperature and evaluation sets.
- **Choosing**: TF-IDF + LogReg (fast, cheap, explainable baseline) → fine-tuned small transformer (best accuracy per cost at scale) → LLM prompting (few labels, many tasks, generation) → RAG when knowledge must come from documents.

CHAPTER 12

NLP Tasks & Evaluation Metrics

Catalogue of tasks • QA • summarisation • machine translation • text generation • chatbots • spelling correction • BLEU • ROUGE • METEOR • BERTScore • perplexity • seqeval • human evaluation

12.1 Task catalogue

Task	Input → output	Classical approach	Modern approach	Metric
Text classification	text → label	TF-IDF + NB/LogReg/SVM	fine-tuned BERT, LLM	accuracy, F1
Sentiment analysis	text → polarity	VADER, TF-IDF + LogReg	BERT/RoBERTa, LLM	F1
NER	tokens → entity tags	CRF	BERT token classification	entity-level F1
POS tagging	tokens → tags	HMM, perceptron	BiLSTM/transformer	accuracy
Question answering	question + context → answer span / text	IR + rules	BERT (extractive), LLM + RAG (generative)	EM, F1
Summarisation	doc → summary	TextRank (extractive)	BART/Pegasus/LLM (abstractive)	ROUGE
Machine translation	text → text in another language	phrase-based SMT	transformer seq2seq, LLM	BLEU, chrF, COMET
Text generation	prompt → text	n-gram LM	GPT-style LLM	perplexity, human eval
Semantic similarity	pair → score	TF-IDF cosine	sentence-transformers	Spearman correlation
Spelling correction	noisy → correct	edit distance + LM (Norvig)	seq2seq	accuracy
Chatbots	dialogue → reply	rules/intents (Rasa)	LLM + RAG + tools	task success, CSAT
Speech (ASR/TTS)	audio ↔ text	HMM-GMM	Whisper, neural TTS	WER

12.2 Generation metrics

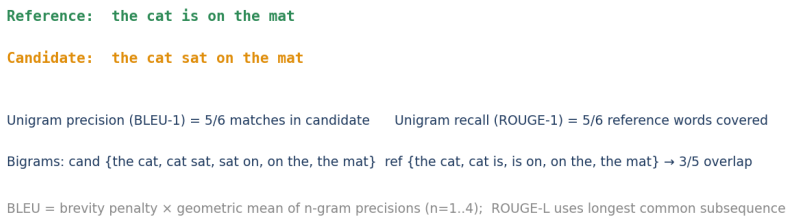


Figure: BLEU is precision-oriented (translation); ROUGE is recall-oriented (summarisation).

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{4} w_n \log p_n\right), \qquad BP = \min(1,\ e^{1-r/c})$$

p_n = clipped n-gram precision, c = candidate length, r = reference length, w_n = 1/4

$$\text{ROUGE-N} = \frac{\sum \text{matching n-grams}}{\sum \text{n-grams in reference}} \qquad \text{WER} = \frac{S+D+I}{N}$$

Metric	Measures	Notes
BLEU	n-gram precision vs reference(s)	MT; corpus-level; insensitive to meaning-preserving paraphrase
ROUGE-1/2/L	n-gram / LCS recall	summarisation
METEOR	unigram matches incl. stems & synonyms	better correlation than BLEU
BERTScore	token embedding similarity	captures paraphrase
COMET	learned MT quality model	modern MT standard
Perplexity	LM fit	lower better
Exact Match / token F1	QA answers	SQuAD
seqeval F1	entity spans	NER
Human / LLM-as-judge	fluency, faithfulness, helpfulness	gold standard / scalable proxy

metrics.py

```
import evaluate
evaluate.load("bleu").compute(predictions=["the cat sat on the mat"], references=[["the cat is on the mat"]])
evaluate.load("rouge").compute(predictions=[summary], references=[reference])
from seqeval.metrics import classification_report; classification_report(y_true_tags, y_pred_tags)
```

CHAPTER 13

Deploying NLP Models

Saving pipelines • FastAPI service • batching • latency & model size • monitoring text drift • library comparison

13.1 Save everything that transforms text

save_pipeline.py

```
import joblib
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer
full = Pipeline([("clean", FunctionTransformer(lambda X: [preprocess(t) for t in X])),
                 ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2)), ("clf", LogisticRegression(max_iter=2000))])
full.fit(X_train, y_train); joblib.dump(full, "ticket_clf.joblib")   # use a named function (not lambda) for pickling in
        production
```

13.2 Serve with FastAPI

api.py

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
app = FastAPI(); model = joblib.load("ticket_clf.joblib")          # load once at startup
class Texts(BaseModel): texts: list[str]
@app.post("/classify")
def classify(req: Texts):
    probs = model.predict_proba(req.texts)
    return [{"label": model.classes_[p.argmax()], "confidence": float(p.max())} for p in probs]
```

13.3 Production concerns

Concern	Practice
Latency	TF-IDF models: sub-millisecond; transformers: batch requests, ONNX Runtime, quantisation, DistilBERT/MiniLM, GPU
Consistency	identical preprocessing in training and serving; version the vocabulary/tokenizer
Confidence	route low-confidence predictions to humans
Monitoring	OOV rate, text length distribution, label distribution, embedding drift, accuracy on sampled human labels
Retraining	schedule or trigger on drift; keep a golden test set
Privacy	mask PII (names, phone numbers, account numbers) before logging or sending to external APIs
Fairness	check performance across languages, dialects, demographics

CHAPTER 14

Interview Questions & Answers

Fundamentals • preprocessing • representation • n-grams & language models • Word2Vec & embeddings • classification • POS / NER / parsing • deep learning & transformers • scenarios

14.1 Fundamentals & pipeline

Q1. What is NLP? NLU vs NLG?

NLP enables computers to process human language. NLU extracts meaning (intent, entities, sentiment); NLG produces fluent text (summaries, replies).

Q2. Explain the end-to-end NLP pipeline.

Data acquisition → cleaning → preprocessing (tokenise, normalise) → feature engineering (BoW/TF-IDF/embeddings) → modelling → evaluation and error analysis → deployment → monitoring and retraining; it is iterative.

Q3. Why is NLP difficult?

Ambiguity at every level (lexical, syntactic, semantic, pragmatic), context dependence, sarcasm, spelling and slang variation, code-mixing, sparsity of rare words and the need for world knowledge.

Q4. What are corpus, document, vocabulary and token?

Corpus = collection of texts; document = one text unit; token = unit after tokenisation; vocabulary = set of unique tokens.

Q5. Heuristic vs ML vs DL approaches?

Heuristics for clear patterns and no data; classical ML with engineered features for small-medium labelled data and interpretability; deep learning/transformers for complex context with more data or pre-trained models.

14.2 Preprocessing

Q6. What is tokenisation and its types?

Splitting text into tokens: sentence, word, subword (BPE/WordPiece/Unigram) and character tokenisation, each trading vocabulary size, OOV handling and sequence length.

Q7. Why lowercase? When not?

To merge case variants and shrink vocabulary; avoid it for NER, acronyms or when case carries meaning (US vs us), and for cased transformer models.

Q8. Stemming vs lemmatisation?

Stemming strips suffixes by rules and can produce non-words (studies → studi); lemmatisation uses a dictionary and POS to return valid base forms (studies → study, better → good); lemmatisation is slower but more accurate.

Q9. Over-stemming vs under-stemming?

Over-stemming merges different words into one stem (universe/university → univers); under-stemming fails to merge related forms (data/datum).

Q10. Why does the WordNet lemmatizer need POS?

Because lemmas depend on the word's role: 'running' as a noun stays 'running', as a verb becomes 'run'; the default POS is noun.

Q11. Should you always remove stop words?

No. Remove them for BoW/TF-IDF topic tasks, but keep negations for sentiment, keep question words for QA, and never remove them for transformers or translation.

Q12. How do you handle emojis, URLs and contractions?

Demojize emojis into words (they carry sentiment), remove or replace URLs/emails with tokens, and expand contractions so negation like 'isn't' → 'is not' is preserved.

Q13. What is Zipf's law?

Word frequency is inversely proportional to its rank, so a few words are extremely frequent and most words are rare — motivating stop-word removal, min_df, and subword tokenisation.

14.3 Representation: BoW, n-grams, TF-IDF

Q14. What is one-hot encoding and its problems?

Each word is a V-dimensional vector with one 1; it is sparse and huge, captures no similarity (all words orthogonal), and cannot represent OOV words.

Q15. Explain Bag of Words.

Represents a document as counts of vocabulary words, ignoring order; simple and effective with linear models but sparse, order-blind and semantics-free.

Q16. Binary BoW vs count BoW?

Binary marks presence/absence; count keeps frequency. Binary often works better for short texts and Bernoulli NB.

Q17. What are n-grams and why use bigrams?

Contiguous sequences of n tokens; bigrams capture local order and phrases like 'not good' or 'New Delhi' that unigrams miss.

Q18. Downside of large n?

Vocabulary explodes and features become sparse, increasing memory and overfitting; use min_df and limit to (1,2) or (1,3).

Q19. Explain TF-IDF with formula.

TF measures how often a term appears in a document, IDF = log(N/df) measures how rare it is across documents; their product highlights terms frequent in a document but rare in the corpus.

Q20. What does a word appearing in every document get?

IDF = log(N/N) = 0 in the textbook formula, so its TF-IDF is 0; sklearn's smoothed idf gives the minimum value 1 before normalisation.

Q21. What is sublinear_tf?

Replacing tf with 1 + log(tf) so repeating a word many times doesn't dominate.

Q22. Why fit the vectoriser only on training data?

Fitting on all data leaks test vocabulary and IDF statistics into training, inflating evaluation scores.

Q23. TF-IDF vs Word2Vec?

TF-IDF is sparse, count-based, interpretable and has no semantics; Word2Vec gives dense learned vectors where similar words are close, generalising to synonyms but losing exact term matching.

Q24. What is HashingVectorizer?

Maps tokens to columns with a hash function, requiring no stored vocabulary — memory-efficient and streamable but with collisions and no inverse mapping.

14.4 Language models

Q25. What is a language model?

A model that assigns probabilities to word sequences or predicts the next word; n-gram LMs use counts with the Markov assumption, neural LMs use networks.

Q26. What is the Markov assumption?

The next word depends only on the previous n−1 words instead of the full history.

Q27. Why smoothing?

Unseen n-grams get zero probability, zeroing the whole sentence; smoothing (Laplace, interpolation, Kneser-Ney) reserves probability for unseen events.

Q28. What is perplexity?

The exponential of the average negative log-likelihood per token — the effective number of choices per word; lower is better.

14.5 Word2Vec & embeddings

Q29. What is a word embedding?

A dense low-dimensional vector learned from context so that words with similar usage have similar vectors.

Q30. What is the distributional hypothesis?

Words that occur in similar contexts tend to have similar meanings — the principle behind Word2Vec and GloVe.

Q31. Explain CBOW.

Takes the context words around a position, averages their embeddings, and predicts the centre word with a softmax; fast and good for frequent words.

Q32. Explain Skip-gram.

Uses the centre word to predict each surrounding context word, creating several training pairs per position; slower but better for rare words and small data.

Q33. CBOW vs Skip-gram — which and when?

CBOW for large corpora and speed; Skip-gram for smaller corpora and better rare-word quality.

Q34. Where are the embeddings in Word2Vec?

In the input-to-hidden weight matrix W (V × N); a one-hot input simply selects a row, which is the word's vector.

Q35. What is negative sampling?

Instead of a full softmax over the vocabulary, train a binary classifier to distinguish the true context word from K randomly sampled noise words, updating only those vectors.

Q36. What is hierarchical softmax?

Computes word probabilities as products of binary decisions along a Huffman tree path, reducing cost from O(V) to O(log V).

Q37. What is subsampling of frequent words?

Randomly discarding very frequent words during training to speed it up and improve vectors of less frequent words.

Q38. Effect of window size?

Small windows capture syntactic/functional similarity; large windows capture topical relatedness.

Q39. What is Average Word2Vec?

A document vector computed as the mean of the Word2Vec vectors of its in-vocabulary words, used as dense features for classifiers.

Q40. Limitations of Average Word2Vec?

Loses word order and negation, dilutes meaning in long documents, ignores OOV words, and treats all words equally — TF-IDF weighting or sequence models address some of this.

Q41. TF-IDF weighted Word2Vec?

A weighted average of word vectors using TF-IDF scores so informative words contribute more than common words.

Q42. Word2Vec vs GloVe vs FastText?

Word2Vec learns from local context windows predictively; GloVe factorises global co-occurrence statistics; FastText adds character n-grams so it can embed OOV and misspelled words.

Q43. How do you handle OOV words?

FastText subwords, subword tokenisers (BPE/WordPiece), an token, character models, or zero vectors in averaging.

Q44. Static vs contextual embeddings?

Static models give one vector per word type; contextual models like BERT produce different vectors depending on the sentence, resolving polysemy.

Q45. Why does king − man + woman ≈ queen?

Consistent relations (like gender) are encoded as roughly constant vector offsets in the embedding space learned from co-occurrence patterns.

14.6 Text classification

Q46. Why is Naive Bayes good for text?

It is fast, needs little data, handles high-dimensional sparse features well, and its independence assumption works surprisingly well for word counts.

Q47. Why Laplace smoothing in Naive Bayes?

Without it a word unseen in a class gives zero probability and wipes out the whole product.

Q48. Why use log probabilities?

Multiplying many small probabilities underflows to zero; summing logs is numerically stable.

Q49. Naive Bayes vs Logistic Regression?

NB is generative and assumes feature independence (good with very little data); LogReg is discriminative and handles correlated features, usually more accurate with more data.

Q50. How would you build a spam classifier?

Clean texts, stratified split, TF-IDF (1,2)-grams in a Pipeline with MultinomialNB or LogisticRegression, tune with grid search on F1, evaluate precision/recall on the spam class, inspect top features, save the pipeline and serve via an API.

Q51. Accuracy is 95% on spam detection — good?

Not necessarily: if 87% is ham, a trivial model gets 87%. Check per-class precision/recall, F1 and the confusion matrix.

Q52. Multi-class vs multi-label?

Multi-class assigns exactly one label (softmax); multi-label allows several (independent sigmoids/one-vs-rest with per-label thresholds).

Q53. How do you handle imbalanced text data?

Stratified splits, class weights, resampling, ComplementNB, threshold tuning and metrics like macro-F1 or PR-AUC.

Q54. Lexicon-based vs ML sentiment?

Lexicon methods (VADER) need no training and are explainable but miss domain language and sarcasm; ML/transformer models learn from labelled data and are more accurate in-domain.

14.7 POS, NER & parsing

Q55. What is POS tagging and why is it useful?

Assigning grammatical categories to tokens; used for lemmatisation, disambiguation, chunking, parsing and as features for NER.

Q56. How does an HMM tagger work?

It models tags as hidden states with transition probabilities P(tag|previous tag) and emission probabilities P(word|tag), both counted from a tagged corpus, and decodes the best tag sequence with Viterbi.

Q57. What is the Viterbi algorithm?

A dynamic programming algorithm that finds the most probable hidden state sequence by keeping, for each position and tag, the best path score and a back-pointer — $O(n \cdot T^2)$.

Q58. What is NER and the BIO scheme?

Identifying and classifying entity spans; BIO tags mark the beginning (B-), inside (I-) and outside (O) of entities so multi-word entities can be labelled token by token.

Q59. Why CRF for NER?

A CRF models the whole tag sequence jointly with rich overlapping features and learns valid transitions, avoiding sequences like O → I-PER.

Q60. Dependency vs constituency parsing?

Dependency parsing links words to their heads with grammatical relations; constituency parsing builds nested phrase structures (NP, VP) from a grammar.

14.8 Deep learning, transformers & scenarios

Q61. Why LSTM over a simple RNN for text?

Gates and an additive cell state let gradients flow over long sequences, solving vanishing gradients so long-range dependencies are learned.

Q62. Why Bidirectional LSTM?

It uses both left and right context for every token, improving classification and tagging when the full text is available.

Q63. How does a CNN work on text?

Filters slide over word embeddings acting as learned n-gram detectors; max pooling keeps the strongest signal; fast and effective for short texts.

Q64. What does the Embedding layer do?

A trainable lookup table mapping token ids to dense vectors, optionally initialised with pre-trained Word2Vec/GloVe.

Q65. Why padding and masking?

Batches need equal-length sequences, so shorter ones are padded; masking tells the model to ignore pad positions.

Q66. What is attention?

A mechanism that computes weights over all input positions so each output focuses on the most relevant tokens; self-attention lets every token attend to every other token.

Q67. BERT vs GPT?

BERT is a bidirectional encoder pre-trained with masked LM, best for understanding tasks; GPT is a causal decoder pre-trained with next-token prediction, best for generation.

Q68. What is WordPiece/BPE and why use subwords?

Subword algorithms build a vocabulary of frequent character sequences so any word can be represented, eliminating OOV while keeping vocabulary small.

Q69. How do you fine-tune BERT for classification?

Tokenise with its tokenizer, add a classification head on the [CLS] representation (AutoModelForSequenceClassification), train with a small learning rate for a few epochs, evaluate with F1.

Q70. BLEU vs ROUGE?

BLEU is precision-based n-gram overlap used for translation; ROUGE is recall-based overlap used for summarisation.

Q71. Your sentiment model fails on 'not bad at all'. Fix?

Keep negations during stop-word removal, add bigrams/trigrams, handle negation scope, or use a sequence/transformer model that understands context.

Q72. You have 300 labelled tickets and 20 categories. What do you do?

Start with an LLM zero/few-shot or embedding + nearest-neighbour approach, use active learning to label more, then train TF-IDF + LogReg or fine-tune a small transformer when data grows; evaluate per category.

Q73. How would you detect duplicate questions?

Sentence embeddings with cosine similarity (or TF-IDF cosine/Jaccard as a baseline) with a tuned threshold; MinHash/LSH for scale; optionally a cross-encoder to verify candidates.

Q74. Classical NLP still relevant in the LLM era?

Yes — TF-IDF/BM25 power search and hybrid RAG, NB/LogReg give cheap fast baselines, and preprocessing, tokenisation and evaluation concepts underpin every LLM system.

CHAPTER 15

Cheat Sheet & Learning Sequence

One-page formulas • which technique when • library one-liners • recommended study order

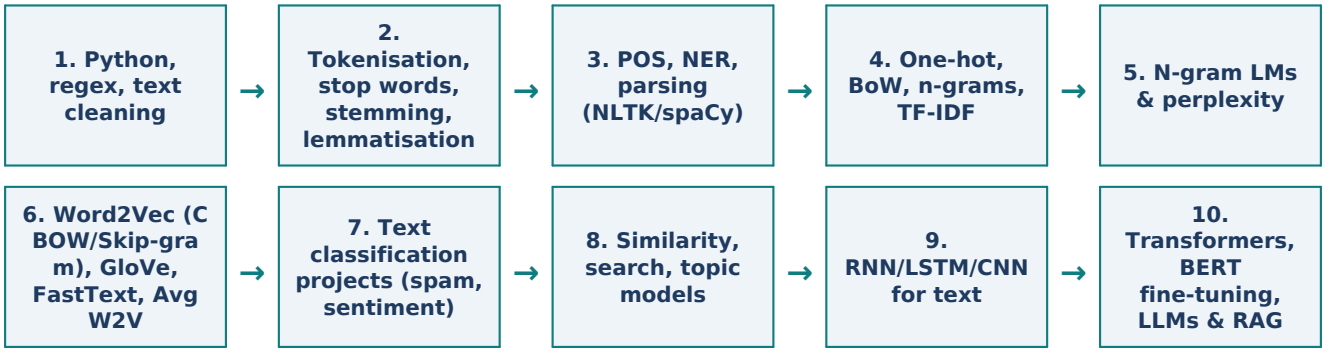
15.1 Formulas

TF-IDF	$\text{tf}(t,d)\cdot\log\frac{N}{\text{df}(t)}$
sklearn idf	$\ln\frac{1+N}{1+\text{df}}+1$
Cosine	$\frac{A\cdot B}{\|A\|\|B\|}$
Bigram LM	$P(w_i\|w_{i-1})=\frac{C(w_{i-1}w_i)}{C(w_{i-1})}$
Laplace	$\frac{C+1}{N+V}$
Perplexity	$\exp\left(-\frac{1}{N}\sum\log p(w_i\|\cdot)\right)$
Naive Bayes	$\arg\max_c \log P(c)+\sum_i\log P(w_i\|c)$
HMM tagging	$\arg\max\prod P(w_i\|t_i)P(t_i\|t_{i-1})$
Skip-gram	$\sum_t\sum_j\log P(w_{t+j}\|w_t)$
Neg. sampling	$\log\sigma(v'^T_o v_i)+\sum_k\log\sigma(-v'^T_k v_i)$
Avg Word2Vec	$\frac{1}{\|d\|}\sum_{w\in d}v_w$
Attention	$\text{softmax}(QK^T/\sqrt{d_k})V$
F1	$\frac{2PR}{P+R}$
BLEU	$BP\cdot\exp(\sum w_n\log p_n)$

15.2 Which technique when?

Situation	Go-to
Quick baseline classifier	TF-IDF (1,2) + LogisticRegression / LinearSVC
Very little data, text classification	Naive Bayes, pre-trained embeddings, LLM few-shot
Need semantics / synonyms	Word2Vec/GloVe averages, sentence-transformers
Typos, OOV, Indian languages	char n-grams, FastText, multilingual transformers
Exact keyword search	BM25
Semantic search / RAG	sentence embeddings + vector DB (+ BM25 hybrid)
Entities	spaCy NER → fine-tuned BERT / LLM extraction
Topics in unlabelled text	LDA / NMF / BERTopic
Best accuracy with labels	fine-tuned transformer
Generation, summarisation, chat	LLMs (+ RAG)

15.3 Recommended learning sequence



Practice projects in order: SMS spam filter (BoW/TF-IDF + NB) → movie/product sentiment (TF-IDF vs Avg Word2Vec vs BiLSTM) → resume/KYC entity extraction (spaCy NER) → FAQ semantic search (TF-IDF → sentence embeddings) → ticket topic discovery (LDA/BERTopic) → fine-tuned DistilBERT classifier served with FastAPI → RAG assistant over documents.

Watch out: Code in this guide is reference-quality but untested in your environment; library APIs (especially transformers, Keras and NLTK data package names) change between versions — check the docs of your installed versions.